

TP N°1: arVault

[75.73/TB034] Arquitectura de Software

CURSO: 01-Calónico

Primer cuatrimestre de 2026

Alumno/a:	FARALL CASADO, Tomás
Número de padrón:	102628
Email:	tcada@fi.uba.ar

Alumno/a:	VACCARELLI, Santiago
Número de padrón:	106051
Email:	svaccarelli@fi.uba.ar

Alumno/a:	JANG, Lucas
Número de padrón:	109151
Email:	ljang@fi.uba.ar

Alumno/a:	KRESTA, Facundo Ariel
Número de padrón:	110857
Email:	facundoarielkresta@gmail.com

7 de mayo de 2026

Índice

1	Introducción	6
1.1	Contexto	6
1.2	Objetivo	6
1.3	Stack tecnológico	6
1.4	Topología de despliegue	6
1.5	Patrón arquitectónico	7
1.6	Endpoints disponibles	7
2	Atributos Importantes	8
2.1	Atributos de calidad clave	8
2.1.1	Seguridad (Security):	8
2.1.2	Disponibilidad (Availability):	8
2.1.3	Desempeño (Performance):	8
2.1.4	Escalabilidad (Scalability):	8
2.1.5	Usabilidad (Usability):	9
2.2	Otros atributos de calidad	9
2.2.1	Visibilidad / Observabilidad (Visibility)	9
2.2.2	Testeabilidad (Testability)	9
2.2.3	Portabilidad (Portability)	9
2.2.4	Interoperabilidad (Interoperability)	9
3	Decisiones de Diseño (C&C)	10
3.1	Evolución: Del Monolito Acoplado a Arquitectura en Capas	10
3.1.1	Mejora propuesta	10
3.1.2	Atributos de calidad afectados	11
3.1.2.1	Testability	11
3.1.2.2	Maintainability	12
3.1.2.3	Modifiability	12
3.1.2.4	Performance (atributo afectado negativamente)	12
3.2	Variables en memoria	12
3.2.1	Mejora propuesta	12
3.2.2	Atributos de calidad afectados	14
3.2.2.1	Scalability	14
3.2.2.2	Performance (atributo afectado negativamente)	14
3.3	Persistencia lazy	14
3.3.1	Mejora propuesta	14
3.3.2	Atributos de calidad afectados	14
3.3.2.1	Durability	14
3.3.2.2	Performance	14
3.4	Persistencia en archivos JSON	14
3.4.1	Mejora propuesta	15
3.4.2	Atributos de calidad afectados	15
3.4.2.1	Availability	15
3.4.2.2	Reliability	15
3.4.2.3	Efficiency	15
3.4.2.4	Scalability	15
3.4.2.5	Testability	15
3.5	Función <code>transfer()</code> mockeada de forma secuencial	15
3.5.1	Atributos de calidad afectados	16
3.5.1.1	User-Perceived Performance	16
3.5.1.2	Efficiency	16
3.5.1.3	Visibility	16

3.5.1.4	Testability	16
3.5.1.5	Interoperability	16
3.6	Instancia única sin redundancia ni healthcheck	16
3.6.1	Modos de falla que quedan silenciosos sin healthcheck	17
3.6.2	Blast radius: cualquier falla = todos los usuarios afectados	17
3.6.3	Configuración de Nginx: mala incluso asumiendo instancia única	17
3.6.4	Docker Compose: lo que falta en la declaración del servicio	18
3.6.5	Tácticas que resuelven este problema	19
3.6.6	Resumen de impacto sobre QA	21
3.7	Sin instrumentación en la aplicación	21
3.8	HTTP 500 para errores de negocio	21
3.9	Validación de entrada por falsiness	22
3.9.1	Atributos de calidad afectados	22
3.9.1.1	Security	22
3.9.1.2	Usabilidad	22
3.9.1.3	Testability	22
3.10	Log de transacciones ilimitado en memoria	23
3.10.1	Atributos de calidad afectados	23
3.10.1.1	Network Performance	23
3.10.1.2	Efficiency	23
3.10.1.3	Availability	23
3.10.1.4	Usabilidad	23
3.11	Impacto de las Decisiones de Diseño sobre los QA	23
3.11.1	Tabla resumen	23
4	Escalabilidad horizontal con Redis	25
4.1	Objetivos	25
4.2	Cambios recomendados en <code>app/core/stateManager.js</code> (conceptual)	25
4.3	Refactor por repositorio (ejemplos)	25
4.3.1	<code>accountRepository</code>	25
4.3.2	<code>rateRepository</code>	25
4.3.3	<code>logRepository</code>	25
4.4	Opciones para operaciones compuestas y atomicidad	26
4.5	Cambios en <code>docker-compose.yml</code> y Redis	26
4.6	Cambios en Nginx (reverse proxy) para balanceo	26
4.7	Healthchecks y readiness	27
4.8	Pruebas y validación	27
4.9	Checklist mínimo para desplegar múltiples réplicas (resumen corto)	27
4.10	Recursos adicionales	27
5	Tácticas de mejora	28
5.1	Tácticas elegidas y descartadas	28
5.2	Requisitos previos	28
5.3	Metodología de medición	28
5.3.1	Escenarios usados	29
5.3.2	Qué medimos	29
5.3.3	Paquete A — Detección: <code>/health + HEALTHCHECK</code>	29
5.3.3.1	Problema	29
5.3.3.2	Táctica aplicada	29
5.3.3.3	Implementación	29
5.3.3.4	Endpoint	29
5.3.3.5	Docker HEALTHCHECK	29
5.3.3.6	Cambios en <code>app/app.js</code>	30
5.3.3.7	Cambios en <code>app/Dockerfile</code>	30

5.3.3.8	Cambios en <code>docker-compose.yml</code>	30
5.3.3.9	Verificación	30
5.3.3.10	Resultados y Métricas	30
5.3.3.11	Propósito	30
5.3.4	Paquete B — Recuperación: <code>restart: unless-stopped</code>	31
5.3.4.1	Problema	31
5.3.4.2	Táctica aplicada	31
5.3.4.3	Implementación	31
5.3.4.4	Cambio en <code>docker-compose.yml</code>	31
5.3.4.5	Cambio realizado	31
5.3.4.6	MTTR esperado	31
5.3.4.7	Verificación	31
5.3.4.8	Resultados y Métricas	32
5.3.5	Paquete C — Nginx: <code>keepalive + timeout</code>	32
5.3.5.1	Problema	32
5.3.5.2	Táctica aplicada	32
5.3.5.3	Implementación	32
5.3.5.4	Cambios en <code>nginx.reverse-proxy.conf</code>	32
5.3.5.5	Cambios clave	33
5.3.5.6	Verificación	33
5.3.5.7	Impacto Esperado	33
5.3.5.8	Resultados Medidos: Sustained Load Test	33
5.3.6	Consolidación de Cambios	34
5.3.6.1	Archivos modificados	34
5.3.6.2	Docker Stack Status	34
5.3.7	Métricas de Éxito	34
5.3.7.1	Paquete A: Detecta estado real	34
5.3.7.2	Paquete B: Recupera automático	34
5.3.7.3	Paquete C: Reduce TCP overhead	34
5.3.8	Trazabilidad	35
5.4	Paquetes descartados: justificación	35
5.4.1	Paquete D — Límites de recursos (<code>mem.limit</code> , <code>cpus</code>)	35
5.4.2	Paquete E — Redundancia activa (múltiples réplicas)	35
5.5	Cierre	35
6	Pruebas	36
6.0.1	Correr los escenarios	36
6.0.1.1	Escenarios disponibles	36
6.0.1.2	Workloads disponibles	36
6.1	Objetivo	36
6.2	Workload Models (Modelos de Carga)	36
6.2.1	Baseline (Smoke / Low-Constant)	36
6.2.2	Load Test (Carga Normal Sostenida)	37
6.2.3	Stress Test (Buscar el Punto de Quiebre)	37
6.3	Escenarios de Artillery a Implementar	37
6.3.1	<code>perf/rates.yaml</code> (dado por la cátedra)	37
6.3.2	<code>perf/exchange.yaml</code> (nuevo)	37
6.3.3	<code>perf/mixed.yaml</code> (nuevo)	37
6.4	Plan de Ejecución	38
6.4.1	Fase A – Baseline	38
6.4.2	Fase B – Redis + arquitectura de capas + <code>healthcheck</code> + <code>keepalive</code>	38
6.4.3	Fase C – Escalado horizontal (3 instancias)	38
6.5	Criterios de Éxito de Esta Fase	38
6.6	Análisis de resultados	39

6.6.1	Escenario ‘rates’ (GET /rates)	39
6.6.1.1	Baseline	39
6.6.1.2	Load	40
6.6.1.3	Stress	41
6.6.1.4	Conclusión	42
6.6.2	Escenario ‘exchange’ (POST /exchange)	42
6.6.2.1	Baseline	42
6.6.2.2	Load	43
6.6.2.3	Stress	44
6.6.2.4	Conclusión	45
6.6.3	Escenario ‘mixed’ (mix 60 % rates / 30 % exchange / 5 % accounts / 5 % log)	45
6.6.3.1	Baseline	45
6.6.3.2	Load	46
6.6.3.3	Stress	47
6.6.3.4	Conclusión	48
6.7	Puntos De Ruptura Identificados	48
6.7.1	‘/rates’ (Read-Only)	48
6.7.2	‘/exchange’ (Business Logic)	49
6.7.3	‘/mixed’	49
6.8	Análisis Profundo: ¿Por Qué Falla?	49
6.8.1	Latencia Base de ‘/exchange’	49
6.8.2	Single-Threaded Event Loop	49
6.8.3	Race Condition	50
6.9	Fase B (Redis + arquitectura de capas + healthcheck + keepalive)	50
6.9.1	Resumen rápido	50
6.9.2	Análisis: ¿Por qué errores en mixed y por qué exchange es cuello de botella?	50
6.9.3	Respuestas por escenario (Fase 2)	50
6.9.3.1	Escenario rates	50
6.9.3.2	Escenario exchange	51
6.9.3.3	Escenario mixed	52
6.10	Fase C – Escalado horizontal (3 instancias)	53
6.10.1	Archivos generados	53
6.10.2	Resumen ejecutivo	53
6.10.3	Respuestas por escenario (Fase 3)	53
6.10.3.1	exchange – Load	53
6.10.3.2	exchange – Stress	54
6.10.3.3	mixed – Load	54
6.10.3.4	mixed – Stress	54
6.10.4	Comparación con Fase B	54
6.10.5	Análisis: ¿Por qué desaparecieron los errores 5xx/ETIMEDOUT?	54
6.10.6	Hallazgos clave (evidencia)	54
6.11	Conclusión	55
6.12	Formato de las respuestas	55
6.13	Escenario rates (GET /rates)	55
6.13.1	Baseline	55
6.13.2	Load	55
6.13.3	Stress	55
6.13.4	Conclusión rates	56
6.14	Escenario exchange (POST /exchange)	56
6.14.1	Baseline	56
6.14.2	Load	56
6.14.3	Stress	56
6.14.4	Conclusión exchange	56
6.15	Escenario mixed (mix 60 % rates / 30 % exchange / 5 % accounts / 5 % log)	56

6.15.1	Baseline	56
6.15.2	Load	56
6.15.3	Stress	57
6.15.4	Conclusión mixed	57
6.16	Recomendaciones generales (prioritizadas)	57
7	Conclusión	58
8	Crítica Global	58
8.1	Lo que se hizo bien	58
8.2	Problemas críticos	59
8.3	Problemas importantes	59
8.4	Deuda técnica reconocida por el desarrollador	59
8.5	Diagnóstico final	59

1. Introducción

1.1. Contexto

La startup **arVault** es una fintech que opera una billetera digital, de reciente creación. Su fundador, un desarrollador aficionado y entusiasta con algo de dinero y muchas ideas, consciente de que la clave de su negocio es la implementación de su **backend**, tercerizó el desarrollo de las aplicaciones para dispositivos móviles, y se concentró en implementar rápidamente (bajo la consigna *fake it till you make it*¹) un núcleo de servicios que le permitieran conseguir más fondos a través de inversores.

Luego de la ronda inicial, los primeros fondos fueron utilizados, no para robustecer la arquitectura existente, sino para agregar más funcionalidad. Una de estas funcionalidades nuevas permite abrir cuentas en distintas monedas (que se respaldan en bancos existentes), y realizar operaciones de cambio entre éstas. El diferencial de **arVault** es tener la tasa de cambio más conveniente dentro de las aplicaciones que proveen este servicio. Para ganar usuarios, las tasas de cambio no tienen *gap* entre el valor de compra y de venta.

Si bien el lanzamiento fue exitoso, comenzaron a aparecer reclamos de los usuarios sobre problemas en el uso de la función de cambio de monedas. Estos reclamos llegaron en el peor momento, dado que **arVault** necesita captar más inversiones y, debido a la caída de la reputación y las reviews negativas, los potenciales inversores se niegan a aportar más fondos si no se realiza una auditoría y se mejora el servicio.

Frente a este reclamo, **arVault** decide contratar a un grupo de arquitectos para que evalúen, propongan e implementen soluciones que mejoren los atributos de calidad del servicio de cambio de monedas.

1.2. Objetivo

El objetivo de este informe es presentar una evaluación de la arquitectura actual del servicio de cambio de monedas, identificar los atributos de calidad que se ven afectados por los reclamos de los usuarios, y proponer decisiones de diseño que puedan mejorar dichos atributos. Además, se presentarán pruebas y métricas en el para evaluar el impacto de las decisiones propuestas y se discutirán las conclusiones obtenidas a partir del análisis realizado. Finalmente, se incluirá una sección de conclusiones que sintetice los hallazgos y recomendaciones para **arVault**.

1.3. Stack tecnológico

Componente	Tecnología	Versión
API REST	Node.js + Express	22 / 4.21.2
Persistencia	Archivos JSON en disco	—
Contenedores	Docker + Docker Compose	—
Reverse proxy	Nginx	1.29.8
Generación de carga	Artillery + plugin-statsd	2.0.22 / 2.2.1
Métricas	StatsD + Graphite + Grafana	— / 1.1.10-5 / 11.5.10
Monitoreo de contenedores	cAdvisor	0.51.0

1.4. Topología de despliegue

```

Cliente / Artillery (load generator)
  |
  v
+-----+
|  Nginx  | <- Reverse proxy, puerto 5555->80
| (proxy) |   upstream hardcoded: exchange-api-1:3000
+-----+

```

¹Expresión popular que refiere a simular confianza o habilidades hasta realmente adquirirlas.

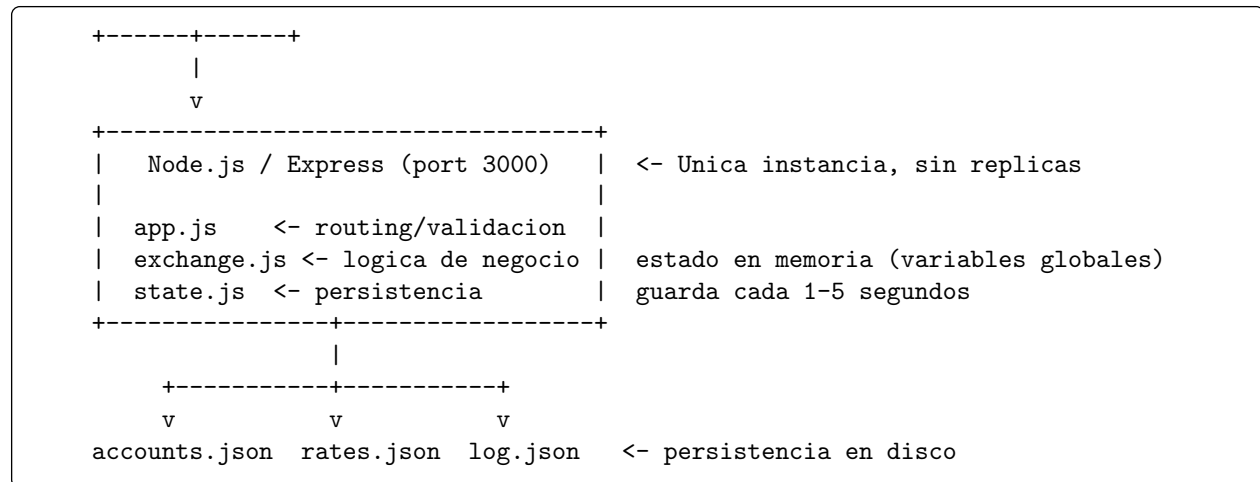


Diagrama 1.1: Arquitectura primera del servicio.

Stack de observabilidad (desacoplado de la app):

```

Artillery --> StatsD --> Graphite <-- cAdvisor
                        |
                        Grafana
(4 paneles: RPS, codigos HTTP,
latencia cliente, CPU/memoria)

```

1.5. Patrón arquitectónico

Monolito de **tres capas** (controller → business logic → persistence) con **estado centralizado en memoria** y **persistencia eventual** en archivos JSON. No hay base de datos relacional, no hay cola de mensajes, no hay mecanismo de coordinación distribuida.

1.6. Endpoints disponibles

Método	Ruta	Descripción
GET	/accounts	Retorna cuentas internas de la empresa
PUT	/accounts/:id/balance	Modifica el saldo de una cuenta interna
GET	/rates	Retorna tasas de cambio vigentes
PUT	/rates	Actualiza una tasa y calcula la recíproca
POST	/exchange	Ejecuta una operación de cambio de monedas
GET	/log	Retorna el historial de transacciones

2. Atributos Importantes

2.1. Atributos de calidad clave

Considerando la naturaleza financiera y transaccional del servicio provisto (*arVault*, un *exchange* de criptomonedas y dinero fiat), se han determinado como centrales los siguientes Atributos de Calidad (QAs):

2.1.1. Seguridad (Security):

Al tratarse de una *fintech* que opera saldos reales en múltiples monedas, la seguridad es el atributo más crítico. Si bien existe un componente externo que gestiona autenticación y autorización, hay aspectos de seguridad que recaen directamente sobre el servicio: la integridad de las transacciones, la prevención de condiciones de carrera que permitan operar con fondos insuficientes y la consistencia del estado financiero. Una brecha en este atributo tiene consecuencias legales, financieras y reputacionales inmediatas. El servicio expone operaciones que modifican saldos reales y tasas de cambio que afectan a todos los usuarios. Sin autenticación ni autorización propias, cualquier actor con acceso de red puede manipular el estado financiero del sistema. Para los inversores que evalúan arVault, una brecha de seguridad evidente es un bloqueante.

2.1.2. Disponibilidad (Availability):

arVault ya sufrió caída de reputación y pérdida de inversores por problemas en el servicio de cambio. Para una billetera digital, la indisponibilidad equivale a pérdida de negocio y confianza de los usuarios. Este atributo es condición necesaria para la continuidad operativa de la empresa. El servicio ya está en producción con usuarios activos. Un tiempo de inactividad tiene impacto directo en reputación, en las reviews negativas que motivan este análisis y en la confianza de los potenciales inversores. En *fintech*, los usuarios asocian la indisponibilidad del servicio con pérdida de dinero o acceso bloqueado a sus fondos, lo que amplifica el daño reputacional.

2.1.3. Desempeño (Performance):

El diferencial de arVault es ofrecer la tasa de cambio más conveniente del mercado. Si las operaciones de *exchange* presentan latencias elevadas, los usuarios perciben el servicio como poco confiable, especialmente en un contexto donde los tipos de cambio fluctúan en tiempo real. El sistema debe procesar múltiples operaciones concurrentes con un tiempo de respuesta adecuado y un alto nivel de *throughput* bajo momentos de mayor demanda. Performance se descompone en tres dimensiones diferenciadas para este servicio:

- **2a — Network Performance:** Eficiencia en el uso de la red entre cliente y servidor (tamaño de respuestas, reutilización de conexiones, compresión).
- **2b — User-Perceived Performance:** Latencia observable por el usuario final desde que emite un request hasta que recibe respuesta. Es el QA de performance más crítico para la experiencia de usuario: determina si arVault se percibe como “rápida” o “lenta”.
- **2c — Efficiency (Resource Efficiency):** Uso eficiente de CPU, memoria y I/O por parte del sistema. Impacta el costo de operación y la degradación del servicio bajo carga sostenida.

El diferencial de arVault es la mejor tasa de cambio del mercado. Si la experiencia es lenta, los usuarios prefieren competidores con tasas levemente peores pero mejor UX. Además, bajo carga concurrente, la ineficiencia del servicio puede amplificar otros problemas (race conditions, I/O blocking).

2.1.4. Escalabilidad (Scalability):

La startup se encuentra en crecimiento activo, captando usuarios e inversores. La arquitectura original fue construida bajo la consigna *fake it until you make it* y no fue diseñada para escalar. Previendo un escenario con cantidad creciente de usuarios operando simultáneamente, el servicio debe poder ser replicado horizontalmente para poder soportar la carga aumentada de usuarios. Si arVault capta inversión y crece en usuarios, la arquitectura debe soportar mayor carga. La escalabilidad determina si el sistema puede crecer

horizontalmente (más instancias) o solo verticalmente (máquina más grande). En un contexto de startup, la escalabilidad horizontal es más económica y resiliente.

2.1.5. Usabilidad (Usability):

El fundador de arVault tercerizó el desarrollo de las aplicaciones móviles, por lo que la usabilidad del frontend queda fuera del alcance directo de este análisis. Se menciona por su importancia, pero no es objetivo de las tácticas a implementar en este trabajo. Los consumidores directos de este servicio son las apps mobile desarrolladas externamente. Una API con semántica incorrecta, mensajes de error crípticos o comportamiento inconsistente obliga a los desarrolladores de las apps a escribir código defensivo complejo y dificulta el diagnóstico de problemas en el cliente.

2.2. Otros atributos de calidad

Además de los QAs clave, se identifican otros atributos que, aunque no son el foco principal de este trabajo, también son relevantes para la calidad general del sistema:

2.2.1. Visibilidad / Observabilidad (Visibility)

Para diagnosticar los reclamos de usuarios y operar un servicio financiero con confianza, el equipo necesita saber en tiempo real qué está pasando dentro del sistema: tasas de error, latencias, volúmenes, estado de los saldos. Sin visibilidad, los problemas se descubren cuando los usuarios se quejan, no cuando ocurren.

2.2.2. Testeabilidad (Testability)

Sin tests automatizados, cada cambio en el código puede introducir regresiones que solo se descubren en producción. En un servicio que maneja dinero, la falta de testeabilidad es un multiplicador de riesgo: no se puede verificar el comportamiento antes de desplegar.

2.2.3. Portabilidad (Portability)

La capacidad de desplegar el sistema en distintos entornos (desarrollo, staging, producción, CI/CD) sin modificar el código reduce la fricción operativa y previene el síndrome de “funciona en mi máquina”. En una startup que itera rápido, esto es crítico para la velocidad de desarrollo.

2.2.4. Interoperabilidad (Interoperability)

El servicio de cambio no opera en aislamiento: depende de un servicio de transferencias externo (actualmente mockeado), necesita integrarse con vaultSec para autenticación, y eventualmente deberá conectarse con bancos reales. La interoperabilidad determina qué tan fácil es integrar, reemplazar o extender estas dependencias.

3. Decisiones de Diseño (C&C)

3.1. Evolución: Del Monolito Acoplado a Arquitectura en Capas

Originalmente, el servicio constaba de un monolito de una sola capa. Había tres archivos con roles distintos (`app.js` operando de HTTP handler, `exchange.js` manejando lógica de negocio, y `state.js` interactuando con la persistencia en disco) que se encontraban acoplados directamente sin una clara inyección de dependencias ni definición de interfaces seguras. Todo corría en el hilo de un único proceso de Node.js, penalizando *Testability*, *Maintainability* y *Modifiability*. El caso base se ilustra debajo:

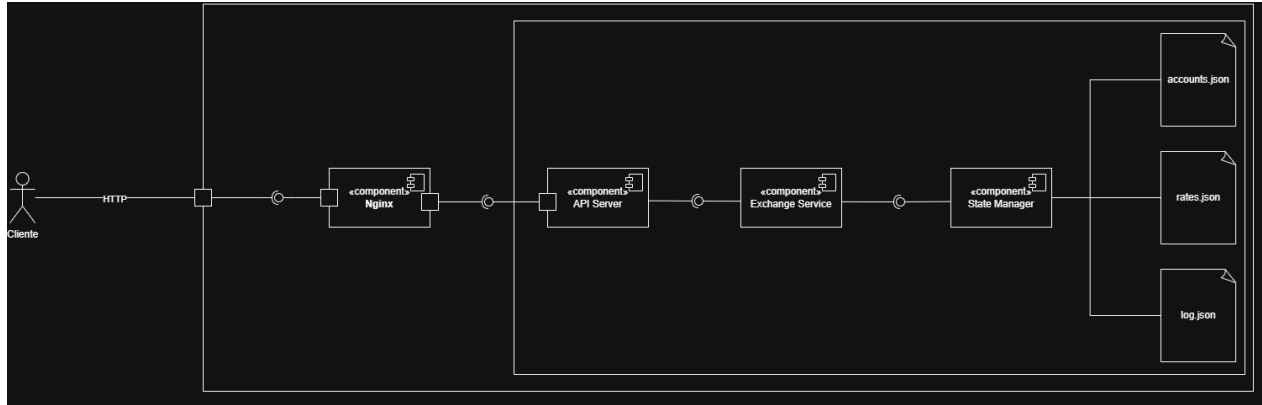


Diagrama 3.1: Arquitectura monolítica del sistema

El **Exchange Service** accede directamente a referencias internas del **State Manager** y muta su estado sin pasar por ninguna interfaz definida, *bypaseando* por completo los métodos expuestos por el **State Manager**.

Además, el **API Server** mezcla responsabilidades: realiza validación de input, invoca la lógica de negocio y decide el formato de respuesta todo en el mismo handler, sin delegar al **Exchange Service** de forma limpia.

3.1.1. Mejora propuesta

La arquitectura propuesta introduce una separación en capas con responsabilidades bien delimitadas. El **API Server** delega cada request a uno de cuatro controladores especializados: **Account Controller**, **Rate Controller**, **Exchange Controller** y **Log Controller**, que se encargan exclusivamente de la validación de input y el mapeo de respuestas HTTP. Cada controlador invoca a su correspondiente servicio de dominio (**Account Service**, **Rate Service**, **Exchange Service** y **Log Service**), donde reside la lógica de negocio. Los servicios acceden al estado únicamente a través de repositorios (**Account Repository**, **Rate Repository**, **Log Repository**), que exponen interfaces limpias y son los únicos componentes autorizados a interactuar con el **State Manager**. Este último centraliza toda la lectura y escritura a disco, persistiendo el estado en `accounts.json`, `rates.json` y `log.json`.

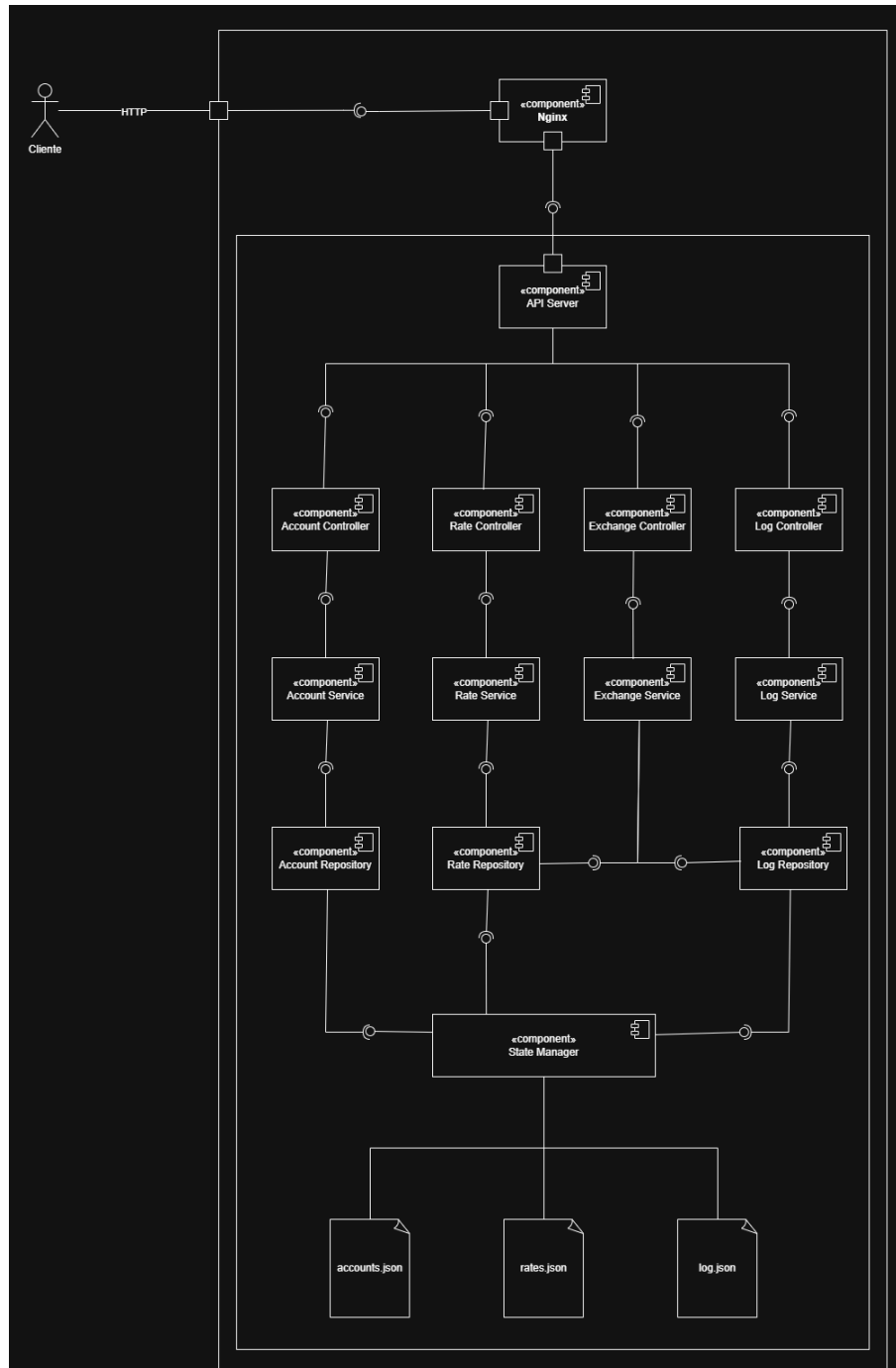


Diagrama 3.2: Arquitectura luego de la mejora

3.1.2. Atributos de calidad afectados

3.1.2.1 Testability

Es el atributo más beneficiado por este cambio. En la arquitectura original, testear el **Exchange Service** implicaba activar toda la cadena de componentes, ya que no había separación entre lógica de negocio, acceso a datos y manejo de estado. En la nueva arquitectura, cada capa puede probarse de forma aislada: los Controllers pueden testearse con mocks de los Services, los Services con mocks de los Repositories, y los Repositories con mocks del **State Manager**. Esto permite pruebas unitarias verdaderas y facilita la detección temprana de

bugs.

3.1.2.2 Maintainability

También mejora considerablemente. La separación por capas y por responsabilidades hace que encontrar a nivel de código dónde está cada cosa sea más fácil e intuitivo, sin tener que recorrer código ajeno a lo que se busca. Si surge un bug, el patrón establecido lo hace directo de encontrar; cada feature nuevo sigue la misma estructura.

3.1.2.3 Modifiability

El cambio tiene un impacto positivo significativo. Con la arquitectura anterior, agregar un nuevo dominio o modificar uno existente podía requerir alterar componentes compartidos como el **Exchange Service**, propagando el riesgo de cambio por toda la cadena. En la nueva arquitectura, cada dominio es una columna vertical independiente (**Controller** → **Service** → **Repository**), de modo que agregar un nuevo dominio implica simplemente agregar una nueva columna sin tocar las existentes. Del mismo modo, cambiar la fuente de datos de los logs, por ejemplo, solo requiere modificar el **Log Repository**, sin afectar al **Log Service** ni al **Log Controller**. Esto reduce el costo y el riesgo de cada modificación futura.

3.1.2.4 Performance (atributo afectado negativamente)

Si bien en teoría la nueva arquitectura introduce más niveles de indirección y por lo tanto podría esperarse un impacto negativo en la latencia, en la práctica las métricas no mostraron diferencias significativas respecto a la arquitectura anterior, o las diferencias registradas fueron lo suficientemente pequeñas como para considerarse despreciables. El overhead generado por los saltos adicionales entre capas no se tradujo en una degradación observable del sistema.

3.2. Variables en memoria

Accounts, rates y log son variables JavaScript dentro del proceso. Leer es instantáneo (bueno para *performance*), pero hace imposible escalar horizontalmente porque cada instancia tendría su propio estado desincronizado.

3.2.1. Mejora propuesta

Se incorporó Redis como almacenamiento centralizado. El estado deja de vivir dentro del proceso y pasa a una fuente de verdad única y compartida entre todas las instancias, eliminando la desincronización.

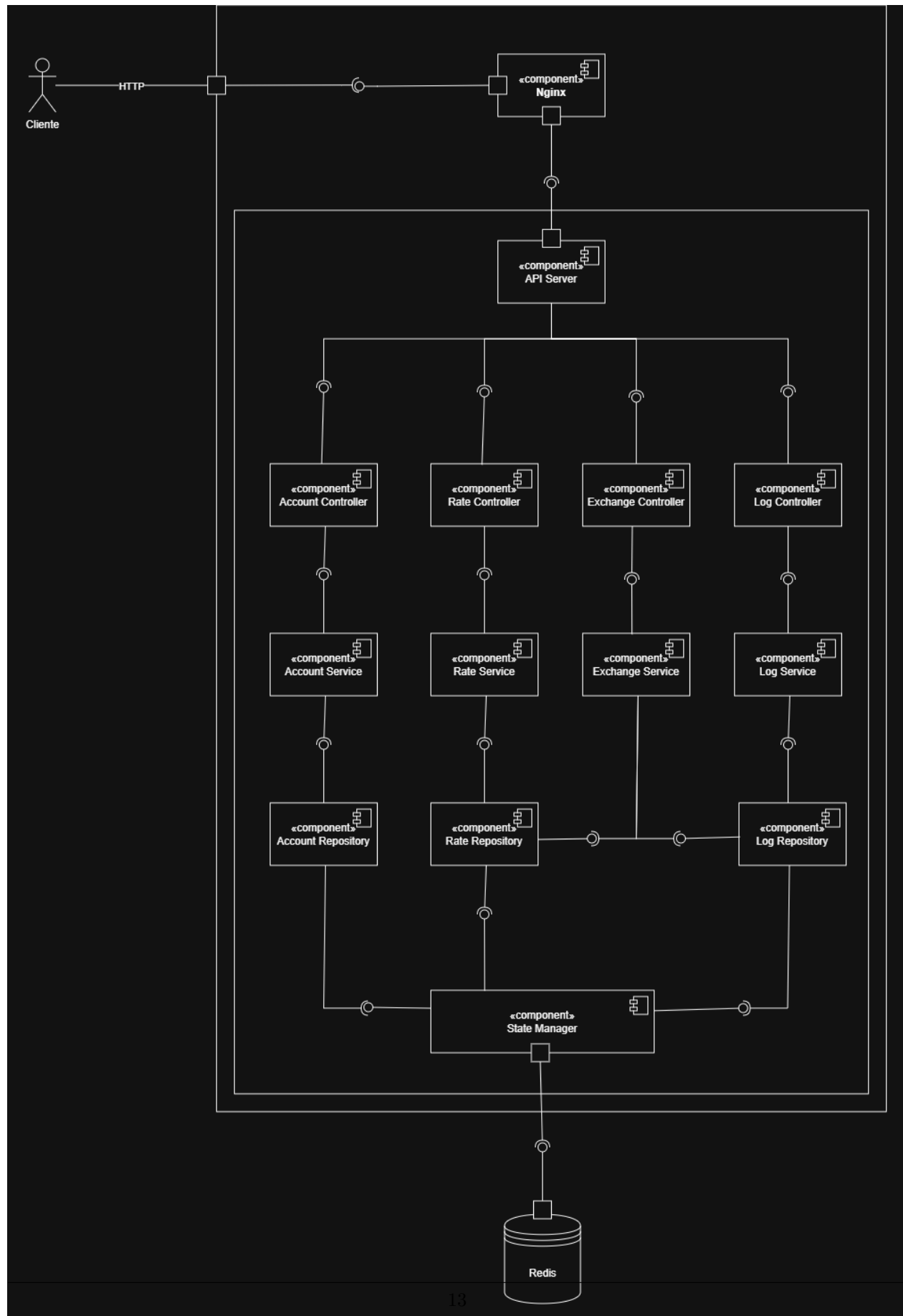


Diagrama 3.3: Arquitectura con Redis como almacenamiento centralizado. Las tres decisiones de diseño que siguen —variables en memoria, persistencia lazy y persistencia en archivos JSON— fueron resueltas todas juntas mediante la incorporación de Redis. El **State Manager** ya no persiste en archivos locales sino que delega en Redis, que actúa como fuente de verdad única y compartida entre todas las instancias.

3.2.2. Atributos de calidad afectados

3.2.2.1 Scalability

El atributo más impactado. Antes era imposible levantar múltiples instancias porque cada una tendría su propio estado desincronizado; con Redis esto queda resuelto.

3.2.2.2 Performance (atributo afectado negativamente)

Leer una variable en memoria es instantáneo, mientras que cada operación contra Redis implica una llamada de red. En la práctica esta latencia suele ser de un par de milisegundos y resulta aceptable, pero es un costo real que antes no existía. En nuestras pruebas la diferencia en performance era despreciable, por lo que el trade-off entre escalabilidad y latencia es favorable.

3.3. Persistencia lazy

El estado se guarda a disco cada 1-5 segundos en lugar de en cada operación. Simplifica el código y no bloquea requests con I/O, pero si el proceso muere entre guardados se pierden transacciones, violando la durabilidad.

3.3.1. Mejora propuesta

Con la incorporación de Redis se removió la persistencia lazy a disco. Redis se encarga de la durabilidad y persistencia de los datos, eliminando la ventana de pérdida de transacciones en caso de caída del proceso.

3.3.2. Atributos de calidad afectados

3.3.2.1 Durability

La ventana de pérdida de datos que generaba el guardado cada 1-5 segundos desaparece. Si el proceso muere, las transacciones están garantizadas porque Redis las persistió antes de confirmarlas.

3.3.2.2 Performance

Dependiendo del nivel de durabilidad configurado en Redis, puede agregarse algo de latencia por operación. Es un costo generalmente aceptable frente a la garantía que ofrece, pero es un trade-off a tener en cuenta.

3.4. Persistencia en archivos JSON

Archivos: `app/state.js:16--23`, `app/state.js:55--68`

```
// Carga inicial desde disco
accounts = await load(ACCOUNTS); // state.js:17
rates    = await load(RATES);    // state.js:18
log      = await load(LOG);      // state.js:19

// Guardado periódico asíncrono
scheduleSave(accounts, ACCOUNTS, 1000); // cada 1 segundo
scheduleSave(rates,    RATES,    5000); // cada 5 segundos
scheduleSave(log,      LOG,      1000); // cada 1 segundo
```

Todo el estado vive en archivos JSON locales. No hay escritura atómica (se usa `writeFile` directamente, sin `write-to-temp-then-rename`), no hay transacciones atómicas, el array `log` crece ilimitadamente (`exchange.js:108`) y es imposible compartir estado entre instancias.

3.4.1. Mejora propuesta

Con la incorporación de Redis se reemplazó completamente la persistencia en archivos JSON por un almacenamiento centralizado. Esto resuelve la falta de transacciones atómicas, la imposibilidad de compartir estado entre instancias y el riesgo de corrupción.

3.4.2. Atributos de calidad afectados

3.4.2.1 Availability

Un crash durante `writeFile` (por ejemplo, disco lleno o SIGKILL) dejaba el archivo JSON parcialmente escrito. Al reiniciar, `JSON.parse` lanzaba una excepción en `state.js:45` y el servicio quedaba inutilizable hasta intervención manual. Con Redis este riesgo desaparece, aunque se introduce una nueva dependencia de infraestructura: si Redis no está disponible, el sistema tampoco funciona.

3.4.2.2 Reliability

El riesgo de corrupción de archivos ya no existe. Redis maneja la integridad de los datos de forma mucho más robusta que un archivo JSON que puede quedar en estado inconsistente si el proceso muere en medio de una escritura. Adicionalmente, Redis permite operaciones atómicas, eliminando la *race condition* TOCTOU del doble gasto donde dos requests concurrentes podían leer el mismo balance y pasar ambas el check de fondos simultáneamente.

3.4.2.3 Efficiency

El array `log` crecía ilimitadamente y `scheduleSave` serializaba el array completo cada 1 segundo mediante `JSON.stringify`. Con alta carga sostenida, esta operación escalaba linealmente en CPU y tiempo, compitiendo con el procesamiento de requests. Redis elimina este problema.

3.4.2.4 Scalability

Era arquitecturalmente imposible correr más de una réplica simultánea de la API: dos procesos escribiendo el mismo `accounts.json` en simultáneo producirían corrupción sin ningún mecanismo de coordinación. Con Redis como fuente de verdad es posible escalar horizontalmente agregando mas instancias de la API, todas compartiendo el mismo estado.

3.4.2.5 Testability

Los tests de integración debían preparar y limpiar archivos JSON en disco. Con Redis es posible inyectar estado sin tocar el filesystem, simplificando el setup de tests.

3.5. Función `transfer()` mockeada de forma secuencial

Archivo: `app/exchange.js:114{120`

```
async function transfer(fromAccountId, toAccountId, amount) {
  const min = 200;
  const max = 400;
  return new Promise((resolve) =>
    setTimeout(() => resolve(true), Math.random() * (max - min + 1) + min));
}
```

La función ignora todos sus parámetros y siempre resuelve `true` tras un delay aleatorio de 200–400 ms. Las dos llamadas en `exchange()` son **secuenciales** (líneas 83 y 86: la segunda `await` comienza solo cuando la primera termina).

3.5.1. Atributos de calidad afectados

3.5.1.1 User-Perceived Performance

↓↓: Cada POST /exchange consume un mínimo garantizado de 400 ms y un máximo de ~800 ms solo en espera de transfers, antes de cualquier procesamiento. Bajo carga, los requests se encolan en el event loop y la latencia percibida crece linealmente.

3.5.1.2 Efficiency

↓ Las dos llamadas podrían ser paralelas (`Promise.all`) sin afectar la corrección del flujo, reduciendo la latencia mínima a 200–400 ms. La decisión de hacerlas secuenciales duplica innecesariamente el tiempo de espera.

3.5.1.3 Visibility

↓↓ Las métricas de latencia del dashboard de Grafana reflejan estos 400–800 ms artificiales. Cuando se integre el servicio real de transferencias, las métricas históricas no serán comparables ni útiles como baseline.

3.5.1.4 Testability

↓↓ El camino de falla de la segunda transferencia (rollback en línea 95) nunca se ejecuta porque `transfer()` nunca falla. Ese código existe pero no ha sido testeado. Su corrección bajo condiciones reales es desconocida.

3.5.1.5 Interoperability

↓↓ No existe contrato definido para el servicio de transferencias real. No hay interfaz, tipo de retorno especificado ni manejo de errores diseñado para el caso donde la transferencia falla parcialmente. La integración futura deberá rediseñar esta parte del flujo.

3.6. Instancia única sin redundancia ni healthcheck

Archivos: `docker-compose.yml` (completo), `nginx_reverse_proxy.conf`

```
# nginx_reverse_proxy.conf
upstream api {
    server exchange-api-1:3000;    ← único upstream, hardcodeado
}
server {
    listen 80;
    location / {
        proxy_pass http://api/;
        # sin timeouts, sin retry, sin health, sin keepalive
    }
}

# docker-compose.yml | sin réplicas, sin healthcheck, sin restart policy
services:
  api:
    build: ./app
    # sin deploy.replicas
    # sin healthcheck
    # sin restart: unless-stopped
    # sin límite de memoria
    # sin límite de CPU
```

Esta es una de las decisiones de diseño con más superficie de impacto. A continuación se detallan los modos de falla específicos, el blast radius y los QA afectados.

3.6.1. Modos de falla que quedan silenciosos sin healthcheck

Un healthcheck distingue entre “proceso corriendo” y “proceso respondiendo correctamente”. Sin él, Docker y Nginx solo saben si el PID 1 del contenedor está vivo. Los siguientes modos de falla dejan el contenedor en estado “running” pero el servicio efectivamente caído:

1. **Event loop bloqueado por operación síncrona.** Node.js es single-threaded. Una operación síncrona costosa (e.g., `JSON.stringify` sobre un log de millones de entradas en `state.js:58`) congela el procesamiento de requests durante segundos. Docker reporta el contenedor “running” durante todo ese tiempo; Nginx sigue enviando requests que se encolan en el socket TCP hasta timeout.
2. **GC thrashing por crecimiento del heap.** El array log crece sin límite (`exchange.js:108`). Antes de que V8 lance OOM, entra en ciclos de garbage collection cada vez más largos y frecuentes (“GC pause”), degradando la latencia sin crashear. Este estado puede durar minutos antes del crash terminal.
3. **Fallo silencioso de scheduleSave.** Los `setInterval` en `state.js:21{23}` ejecutan `save()` sin manejo de error en caso de que `writeFile` falle repetidamente (disco lleno, permisos, corrupción del FS). El error se loguea a `console.error` (`state.js:60`) y el intervalo continúa reintentando. El estado en memoria diverge del estado en disco sin que nadie se entere.
4. **Archivo JSON corrupto tras crash.** `state.js:58` usa `writeFile` directo (no atómico). Un crash durante la escritura deja el archivo con JSON parcial. En el próximo restart, `load()` en `state.js:45` lanza en `JSON.parse`, el catch en línea 46 loguea el error y retorna `undefined`. El servicio queda con `accounts = undefined` pero “running”. El primer request a `/exchange` produce un `TypeError` no capturado.
5. **Inicialización larga.** Al arrancar, `app.js:13` ejecuta `await exchangeInit()` que a su vez lee los tres archivos JSON secuencialmente (`state.js:17{19}`). Con un log.json grande, esto puede tomar varios segundos. Durante ese intervalo el puerto 3000 no escucha aún. Nginx, sin healthcheck, intenta proxear requests al upstream y obtiene `connection refused` → 502.

3.6.2. Blast radius: cualquier falla = todos los usuarios afectados

Al existir una única instancia, la falla no puede ser parcial:

- No hay **isolation entre usuarios**: un usuario que dispare un input que causa `TypeError` (sección 3.8) crasha el proceso y afecta a todos.
- No hay **isolation entre endpoints**: `GET /log` serializa todo el log y bloquea el event loop → `POST /exchange` de otro usuario se queda esperando.
- No hay **circuit breaker**: si el servicio externo de transferencias (cuando se integre real en reemplazo del mock de `exchange.js:114`) se degrada, todos los `POST /exchange` se encolan en el event loop aumentando la memoria y eventualmente tumbando el proceso.
- No hay **bulkhead**: los requests de lectura (`GET /rates`, relativamente baratos) comparten el mismo event loop que los requests pesados (`POST /exchange` con sus 400–800 ms). Bajo carga del segundo, los primeros se degradan igual.

3.6.3. Configuración de Nginx: mala incluso asumiendo instancia única

El bloque `upstream` tiene carencias que amplifican el problema aún con una sola instancia:

lo que falta en `nginx_reverse_proxy.conf`:

```
upstream api {
    server exchange-api-1:3000
        # ausente: Nginx no marca el upstream como down
        max_fails=3 fail_timeout=30s;
    # ausente: cada request abre conexión TCP nueva al upstream
```

```

    keepalive 32;
}

server {
    listen 80;
    location / {
        proxy_pass http://api/;
        # ausente: si el upstream cuelga, el cliente espera el default (60s)
        proxy_connect_timeout 5s;
        # ausente
        proxy_read_timeout 10s;
        # ausente: no hay retry hacia otro upstream (no hay otro, pero tampoco lógica de reintento)
        proxy_next_upstream error timeout;

        # ausente: necesario para keepalive con el upstream
        proxy_http_version 1.1;
        # ausente: necesario para keepalive
        proxy_set_header Connection "";

        # Sin headers informativos al upstream:
        # proxy_set_header X-Real-IP $remote_addr;
        # proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

```

Consecuencias concretas:

- Sin `keepalive`, cada request entrante a Nginx abre y cierra una conexión TCP nueva contra el contenedor de la API. En una prueba de carga sostenida, esto satura la tabla de sockets (`TIME_WAIT`) del kernel mucho antes que la app realmente se quiebre.
- Sin `proxy_read_timeout`, un request `POST /exchange` que quede colgado por un `await transfer()` que nunca resuelve (en una integración real con un banco lento) ocupa un worker de Nginx hasta el default de 60 segundos.
- Sin `proxy_next_upstream` ni backends alternativos, cualquier error del upstream es un 502 al cliente sin mitigación.
- Sin `X-Forwarded-For`, el log de aplicación (si existiera uno real) no puede identificar la IP original del cliente.

3.6.4. Docker Compose: lo que falta en la declaración del servicio

lo que debería tener `services.api` en `docker-compose.yml`:

```

api:
  build: ./app
  # ausente: Docker no reinicia el contenedor si muere
  restart: unless-stopped
  deploy:
    ausente: permitiría escalar horizontalmente
    replicas: 3
  resources:
    limits:
      # ausente: sin límite, el contenedor puede agotar la RAM del host
      memory: 512M
    # ausente

```

```
    cpus: "1.0"
# ausente completamente
healthcheck:
  test: ["CMD", "wget", "-q", "-O-", "http://localhost:3000/health"]
  interval: 10s
  timeout: 3s
  retries: 3
  start_period: 15s
```

Consecuencias concretas:

- Sin **restart: unless-stopped**, un crash del proceso Node.js deja el contenedor en estado “exited” hasta intervención manual. El sistema queda caído indefinidamente.
- Sin endpoint de health en la aplicación (**/health** o **/ready**), no hay manera de implementar el health-check de Docker aunque se quisiera. El servicio debe exponer un endpoint trivial que verifique al menos: proceso vivo, estado cargado (**accounts !== undefined**), y opcionalmente, acceso al filesystem de estado.
- Sin límite de memoria, el memory leak del log crece hasta consumir toda la RAM del host, afectando también a los otros servicios (graphite, grafana, nginx).

3.6.5. Tácticas que resuelven este problema

Ordenadas por costo de implementación:

1. **Endpoint de health** (**/health**) en la app + **healthcheck** en **docker-compose.yml**. Costo bajo, permite detectar el estado degradado antes de impactar al usuario.
2. **restart: unless-stopped** en el servicio api. Costo nulo, mitiga el MTTR ante crashes.
3. **Límites de recursos** (**mem.limit**, **cpus**) para contener el blast radius del memory leak.
4. **Keepalive y timeouts en Nginx** para evitar la saturación de sockets y liberar workers ante upstreams lentos.
5. **Node.js cluster** (módulo **cluster** o **PM2**) para aprovechar múltiples CPUs dentro del mismo contenedor. No resuelve el problema de SPOF ni el de state sharing, pero mejora la capacidad de un nodo único.
6. **Múltiples réplicas + state externalizado (Redis)** para escalado horizontal real. Es la solución definitiva pero requiere refactorizar **state.js** para eliminar el estado en memoria por proceso.

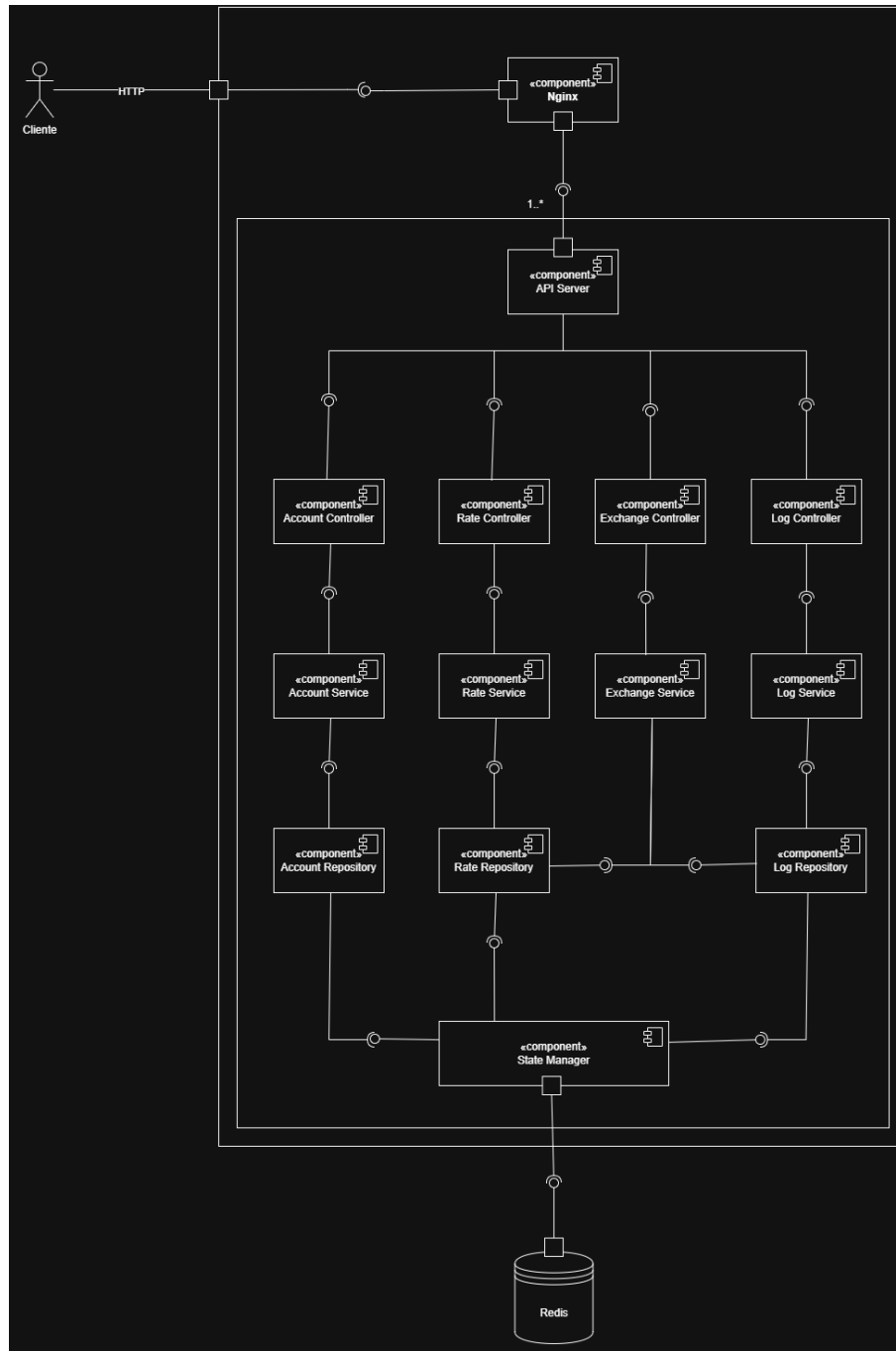


Diagrama 3.4: El único cambio respecto al diagrama anterior es la multiplicidad 1..* entre **Nginx** y la aplicación: ahora pueden existir múltiples instancias, cada una con su propia cadena de Controllers, Services y Repositories, todas compartiendo el mismo estado en Redis.

Las tácticas 1–4 son mejoras quirúrgicas con impacto inmediato sobre Availability y Performance. La táctica 6 es la única que escala Scalability más allá de un solo nodo, pero está bloqueada por la decisión 3.1 (persistencia en JSON local).

3.6.6. Resumen de impacto sobre QA

QA	Impacto	Razón principal
Availability	↓↓↓	SPOF + sin healthcheck + sin restart policy + sin límites de recursos
User-Perceived Performance	↓↓	Event loop único compartido por todos los requests; sin keepalive
Efficiency	↓	Sin keepalive, cada request abre TCP nuevo; GC thrashing invisible
Scalability	↓↓↓	Upstream hardcodeado + estado local impiden réplicas
Visibility	↓	Sin healthcheck no se distingue “running” de “healthy”; modos de falla invisibles
Portability	↓	Nombre de contenedor generado por Compose hardcodeado en Nginx
Testability	↓	Sin endpoint de health, los tests smoke post-deploy no tienen contra qué verificar

3.7. Sin instrumentación en la aplicación

Archivos: `app/app.js`, `app/exchange.js` (sin ningún cliente de StatsD)

El stack de observabilidad (StatsD → Graphite → Grafana) existe, pero la aplicación no está instrumentada. No hay ningún cliente de StatsD en el código Node.js. Los únicos datos que llegan a Grafana son:

- **Artillery:** métricas del generador de carga (latencia del cliente, RPS, códigos HTTP).
- **cAdvisor:** métricas de contenedores (CPU, memoria del proceso completo).

Impacto sobre Visibility ↓↓↓: No se captura ninguna métrica de negocio ni de comportamiento interno:

- No se mide el número de exchanges exitosos vs. fallidos por motivo (fondos insuficientes, error de sistema).
- No se mide el volumen operado por par de monedas (ARS/USD, USD/EUR, etc.).
- No se mide la evolución de los saldos de las cuentas internas.
- No se detecta cuándo un saldo está próximo a agotarse (riesgo de parar operaciones).
- No se puede distinguir la latencia de la lógica de negocio de la latencia de la capa de red.
- Los errores de la aplicación solo aparecen en `console.error`, no en ningún sistema de alertas.

El panel de Grafana muestra latencia y RPS, pero todo medido desde el generador de carga, no desde el servidor. Una degradación interna del servicio (por ejemplo, I/O lento al guardar el log) sería invisible hasta que impacte en la latencia cliente.

3.8. HTTP 500 para errores de negocio

Archivo: `app/app.js:88{92`

```
if (exchangeResult.ok) {
  res.status(200).json(exchangeResult);
} else {
  res.status(500).json(exchangeResult); // fondos insuficientes → 500
}
```

Una operación rechazada por fondos insuficientes retorna HTTP 500 Internal Server Error, el mismo código que un crash del servidor.

Impacto sobre Visibility ↓↓: El panel “Requests State” del dashboard de Grafana agrupa en “Errors” tanto los crashes de infraestructura como los rechazos de negocio normales. Es imposible saber si la tasa de error aumentó porque el servidor está fallando o porque los usuarios se quedan sin fondos. Las alertas sobre ese panel son inherentemente ruidosas e inaccionables.

Impacto sobre Usabilidad ↓↓↓: Las apps mobile que consumen esta API deben tratar el `ok: false` dentro del body JSON para distinguir el rechazo de negocio del error de servidor, ya que el código HTTP no lo informa correctamente. La mayoría de los frameworks HTTP cliente (y las apps de los usuarios) interpretan un 500 como “el servidor rompió”, no como “no tenés fondos”, generando mensajes de error confusos para el usuario final. El código correcto sería HTTP 422 (Unprocessable Entity) para rechazos de negocio.

3.9. Validación de entrada por falsiness

Archivo: `app/app.js:75{83}` (y equivalentes en otros endpoints)

```
if (!baseCurrency || !counterCurrency || !baseAccountId || !counterAccountId || !baseAmount) {  
  return res.status(400).json({ error: "Malformed request" });  
}
```

La validación verifica únicamente que los campos no sean falsy en JavaScript (`null`, `undefined`, `0`, `false`, `NaN`).

Problemas concretos identificados:

1. `PUT /accounts/:id/balance` con `balance: 0` retorna 400 — el operador no puede llevar una cuenta a cero saldo.
2. `baseAmount: -100` es válido — invierte la dirección efectiva del cambio, permitiendo que un usuario “venda” un monto negativo.
3. Si `baseCurrency` no existe en `rates`, la línea `exchange.js:61` ejecuta `rates[baseCurrency][counterCurrency]` donde `rates[baseCurrency]` es `undefined`, lanzando `TypeError: Cannot read properties of undefined`. Express captura la excepción y retorna 500 genérico, sin ningún mensaje útil.
4. El mensaje de error “Malformed request” no indica qué campo falta o tiene formato incorrecto.

3.9.1. Atributos de calidad afectados

3.9.1.1 Security

↓ La ausencia de validación de rango y tipo permite producir estados inválidos mediante inputs deliberadamente malformados (montos negativos, divisas inexistentes).

3.9.1.2 Usabilidad

↓↓ Los mensajes de error no son accionables para el desarrollador de la app mobile. “Malformed request” no dice qué campo es el problema. El crash por `TypeError` retorna 500, haciendo indistinguible un error de validación de un error del servidor.

3.9.1.3 Testability

↓ El comportamiento inconsistente (algunos inputs inválidos retornan 400, otros producen 500 por `TypeError`) hace difícil escribir tests de validación exhaustivos.

3.10. Log de transacciones ilimitado en memoria

Archivos: app/exchange.js:108, app/state.js:19, 23

```
log.push(exchangeResult);           // exchange.js:108 | sin límite ni rotación
scheduleSave(log, LOG, 1000);       // state.js:23 | serializa el array completo cada 1s
```

GET /log retorna el array completo sin paginación ni filtros.

3.10.1. Atributos de calidad afectados

3.10.1.1 Network Performance

⇓ El payload de GET /log crece linealmente con el número de transacciones. En producción con carga sostenida, una respuesta que inicialmente pesa kilobytes puede llegar a megabytes, saturando el ancho de banda del cliente y el tiempo de serialización del servidor.

3.10.1.2 Efficiency

⇓ JSON.stringify(log) ejecutado cada 1 segundo consume CPU y tiempo proporcional al tamaño del array. Con millones de entradas, esta operación compite activamente con el procesamiento de requests, degradando la latencia del sistema completo.

3.10.1.3 Availability

↓ El crecimiento ilimitado del array en memoria puede eventualmente agotar el heap de Node.js. Cuando el proceso supera el límite de memoria del contenedor (sin límite configurado en docker-compose.yml), el OS lo termina y el servicio se reinicia, perdiendo el estado en memoria no persistido.

3.10.1.4 Usabilidad

↓ Sin paginación ni filtros, GET /log es inutilizable para cualquier caso de uso real (auditoría, búsqueda de una transacción específica, consulta de operaciones de un usuario).

3.11. Impacto de las Decisiones de Diseño sobre los QA

3.11.1. Tabla resumen

Cuadro 1: Impacto de decisiones de diseño sobre los atributos de calidad (I): disponibilidad y rendimiento

Decisión de diseño	Avail.	Perf. Red	Perf. UX	Efficiency
Persistencia JSON con save periódico	⇓	—	—	⇓
Estado en memoria sin concurrencia	↓	—	⇓	—
transfer() mockeada secuencial	—	—	⇓	↓
Instancia única, sin réplicas	⇓⇓	—	⇓	—
Sin autenticación propia	—	—	—	—
Sin instrumentación en la app	—	—	—	—
HTTP 500 para errores de negocio	—	—	—	—
Validación por falsiness	—	—	—	—
Log ilimitado en memoria	↓	⇓	↓	⇓
Sin OpenAPI / versionado	—	—	—	—
Nginx upstream hardcodeado	↓	—	—	—

Cuadro 2: Impacto de decisiones de diseño sobre los atributos de calidad (II): escalabilidad, seguridad, mantenibilidad e integración

Decisión de diseño	Scalab.	Security	Visibility	Testability	Portability	Interop.	Usability
Persistencia JSON con save periódico	↓↓↓	—	—	↓	—	—	—
Estado en memoria sin concurrencia	—	—	—	↓↓	—	—	—
<code>transfer()</code> mockeada secuencial	—	—	↓↓	↓↓	—	↓↓	—
Instancia única, sin réplicas	↓↓↓	—	—	—	↓	—	—
Sin autenticación propia	—	↓↓↓	—	—	—	↓	—
Sin instrumentación en la app	—	—	↓↓↓	—	—	—	—
HTTP 500 para errores de negocio	—	—	↓↓	—	—	—	↓↓↓
Validación por falsiness	—	↓	—	↓	—	—	↓↓
Log ilimitado en memoria	—	—	—	—	—	—	↓
Sin OpenAPI / versionado	—	—	—	↓	—	↓↓	↓
Nginx upstream hardcodeado	↓↓	—	—	—	↓↓	—	—

4. Escalabilidad horizontal con Redis

Hoy la app ya utiliza Redis en tiempo de ejecución (seeding desde `state/*.json`), por lo que la barrera principal (persistencia en archivos locales) ya no existe. Sin embargo, las implementaciones actuales hacen read-modify-write de estructuras completas (arrays/objetos serializados) y eso provoca race conditions bajo concurrencia. A continuación se listan los cambios recomendados.

4.1. Objetivos

- Evitar races y pérdida de actualizaciones entre réplicas.
- Usar primitivas atómicas de Redis o transacciones para operaciones concurrentes.
- Cambiar estructuras que se leen/escriben completas (arrays JSON) por estructuras nativas de Redis.

4.2. Cambios recomendados en `app/core/stateManager.js` (conceptual)

- Mantener la inicialización (seed) tal como está; es aceptable que la primera réplica que arranque escriba datos si las claves no existen.
- Asegurarse de exponer un cliente `redis` reutilizable para repositorios.

4.3. Refactor por repositorio (ejemplos)

4.3.1. `accountRepository`

- **Problema actual:** se hace `getAccounts()` → modificar un elemento → `saveAccounts()` (set de todo el array). Bajo concurrencia se pierden actualizaciones.
- **Recomendación:** almacenar cada cuenta como clave/hash Redis.
 - Ejemplo de esquema: `HSET account:{id} id <id>balance <balance>currency <cur>` o `JSON.SET` si se usa Redis.JSON.
 - Para incrementar balance de forma segura: usar `HINCRBYFLOAT account:{id} balance <delta>` o una transacción LUA que haga validaciones y ajustes atómicos.
- **Código ejemplo (pseudocódigo):**

```
// aumentar balance de forma atómica
await redis.hincrbyfloat('account:${id}', 'balance', amount);
// leer cuenta
const data = await redis.hgetall('account:${id}');
```

4.3.2. `rateRepository`

- **Problema actual:** `getRates()` devuelve objeto y `setRate()` reescribe todo el objeto.
- **Recomendación:** usar hashes por base (`HSET rates:{base} {counter} {rate}`) o un hash global con keys compuestas (`HSET rates 'USD:ARS' 350`).

4.3.3. `logRepository`

- **Problema actual:** el log se guarda como array completo; `add()` hace push + set del array.
- **Recomendación:** usar lista Redis con `R PUSH log <json>` y leer con `L RANGE log 0 -1` o usar `L PUSH + L TRIM` para mantener tamaño límite.

4.4. Opciones para operaciones compuestas y atomicidad

- Usar operaciones atómicas de Redis (HINCRBYFLOAT, HSET, RPush) cuando cubran la necesidad.
- Para read-modify-write complejo, usar WATCH + MULTI + EXEC (optimistic locking) o scripts Lua (mejor: ejecutan de forma atómica server-side).
- **Ejemplo LUA para transferencia entre cuentas (debit/credit atómico):**

```
-- args: srcKey, dstKey, amount
local src = KEYS[1]
local dst = KEYS[2]
local amt = tonumber(ARGV[1])
local srcBal = tonumber(redis.call('HGET', src, 'balance') or '0')
if srcBal < amt then return {err='insufficient'} end
redis.call('HINCRBYFLOAT', src, 'balance', -amt)
redis.call('HINCRBYFLOAT', dst, 'balance', amt)
return {ok='ok'}
```

4.5. Cambios en docker-compose.yml y Redis

- **Persistencia:** añadir un volumen para Redis para no perder datos al reiniciar el contenedor.

```
redis:
  image: redis:7
  volumes:
    - redis_data:/data
volumes:
  redis_data:
    driver: local
```

- Escalado local: `docker compose up --scale api=3 -d` (compose v2+). Para producción usar un orquestador (Swarm/Kubernetes).

4.6. Cambios en Nginx (reverse proxy) para balanceo

- No apuntar a una instancia concreta (`exchange-api-1`) sino al nombre de servicio `api` y habilitar resolución dinámica.
- Ejemplo `nginx_reverse_proxy.conf` mínimo:

```
resolver 127.0.0.11 valid=10s;
upstream api {
    server api:3000 resolve;
    keepalive 32;
}

server {
    listen 80;
    location / {
        proxy_pass http://api;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_read_timeout 10s;
    }
}
```

Notas: con `resolve` Nginx consultará el DNS interno de Docker; en entornos reales preferir usar un LB que integre con el orchestrator.

4.7. Healthchecks y readiness

- Mantener el HEALTHCHECK y asegurarse que cada réplica exponga un `/health` correcto (ya corregido para `await`).
- Configurar `nginx` upstream `max.fails/fail.timeout` si hay réplicas para permitir eyección temporal de instancias fallidas.

4.8. Pruebas y validación

- Testear transferencias concurrentes con `small load tests` (Artillery) para detectar races.
- Simular kills (`docker kill --signal=SIGKILL`) y verificar que el sistema mantiene consistencia tras `recovery`.

4.9. Checklist mínimo para desplegar múltiples réplicas (resumen corto)

- ☐ Refactor `accountRepository` a `HSET/HINCRBYFLOAT` o `LUA`.
- ☐ Refactor `rateRepository` a `hashes` (`HSET rates:{base}` o similar).
- ☐ Refactor `logRepository` a `RPUSH + LRANGE` y limitar tamaño con `LTRIM`.
- ☐ Añadir volumen persistente a Redis en `docker-compose.yml`.
- ☐ Cambiar `nginx.reverse.proxy.conf` para usar `resolver` y `server api:3000 resolve;`.
- ☐ Verificar healthchecks por réplica.
- ☐ Ejecutar pruebas de concurrencia y ajustes (`WATCH/MULTI` o `Lua` si se detectan races).

4.10. Recursos adicionales

- Redis transactions: <https://redis.io/docs/manual/transactions/>
- Redis scripting (Lua): <https://redis.io/docs/manual/programmability/eval-intro/>
- Redis data types (Hashes, Lists): <https://redis.io/docs/data-types/>

5. Tácticas de mejora

La sección 3.4 del análisis identifica que el servicio corre como una única instancia sin mecanismos de detección ni recuperación de fallos, y con un reverse-proxy (Nginx) muy básico. Esta guía propone **tres tácticas arquitectónicas concretas** para atacar ese conjunto de problemas, con el protocolo de medición pre/post para mostrar evidencia del impacto.

El alcance se limita a la sección 3.4. Otras mejoras identificadas en el análisis (race condition TOCTOU en 3.2, estado en JSON local en 3.1, log ilimitado en 3.9, autenticación en 3.5, etc.) quedan fuera de esta iteración.

5.1. Tácticas elegidas y descartadas

De las cinco tácticas que surgen del análisis de 3.4, elegimos aplicar las primeras tres:

Paquete	Nombre	Decisión	Motivo
A	Detección: <code>/health</code> + <code>HEALTHCHECK</code>	✓Aplicar	Prerequisito de B. Bajo costo, alto impacto sobre Visibility
B	Recuperación: <code>restart: unless-stopped</code>	✓Aplicar	Una línea en compose; completa el Process Monitor junto con A
C	Nginx: <code>keepalive</code> + <code>timeout</code>	✓Aplicar	Sin tocar código de la app; mejora inmediata en latencia y eficiencia de red
D	Límites de recursos (<code>mem_limit</code> , <code>cpus</code>)	×Descartar	Mejora difícil de evidenciar en una demo corta. Apunta a un problema cuya causa raíz es 3.9 (log ilimitado)
E	Redundancia activa (múltiples réplicas)	×Descartar	Bloqueado por 3.1: con estado en JSON local cada réplica diverge

El cierre del documento (sección 11) justifica con más detalle el descarte de D y E.

5.2. Requisitos previos

Para reproducir el protocolo de medición:

```
# Levantar el stack
docker compose up -d

# Verificar que los servicios están levantados
docker compose ps

# Artillery (desde perf/)
cd perf && npm install
```

Comandos que se usan varias veces en la guía:

```
# Estado del healthcheck de un contenedor (requiere Paquete A)
docker inspect --format '{{.State.Health.Status}}' exchange-api-1

# Ejecutar un escenario Artillery
cd perf && npm run scenario -- <nombre_del_yaml> api
```

5.3. Metodología de medición

La evidencia se arma ejecutando el **mismo escenario Artillery antes y después** de aplicar cada paquete. Pasos:

1. Con el sistema sin el paquete aplicado, correr el escenario y capturar los paneles de Grafana.
2. Aplicar los cambios del paquete (edit + `docker compose up -d --build`).
3. Correr el mismo escenario y capturar los mismos paneles.
4. Llenar la tabla de resultados con pre / post / delta.

5.3.1. Escenarios usados

Escenario	Archivo	Para qué sirve	Paquete
Baseline existente	<code>perf/rates.yaml</code>	Latencia en régimen normal	C
Carga sostenida	<code>perf/sustained.yaml</code> (nuevo)	Expone saturación TCP	C
Chaos kill	<code>perf/chaos-kill.yaml</code> + <code>chaos-kill.sh</code> (nuevos)	Mide MTTR tras matar el proceso	A, B

Los YAML de los dos escenarios nuevos están en la sección 8.

5.3.2. Qué medimos

Métrica	Tipo	De dónde sale
P95 / P99 de latencia	Numérica	Artillery → StatsD → Grafana (panel “Response time”)
Error rate (%)	Numérica	Artillery → StatsD → Grafana (panel “Requests state”)
MTTR tras SIGKILL	Numérica	Ventana de 5xx visible en “Requests state”
Health.Status	Cualitativa	<code>docker inspect</code>

5.3.3. Paquete A — Detección: `/health` + HEALTHCHECK

5.3.3.1 Problema

Como se describe en 3.4.1, hoy Docker no distingue entre “el proceso está vivo” y “el proceso está respondiendo bien”. Si el event loop se bloquea o el estado queda en `undefined` tras un JSON corrupto, el contenedor sigue marcado como `running` y nadie se entera hasta que un usuario se queja.

5.3.3.2 Táctica aplicada

- **Ping/Echo** (Availability – Detección): un componente hace ping y espera un echo en un tiempo predeterminado. Acá el “pinger” es Docker, el “echoer” es el endpoint `/health` de la app.
- **System Monitor** (Availability – Detección): Docker monitorea el contenedor y puede detectar timeouts.
- **Design to be monitored** (Scalability): prerequisite para poder “identify dead services”.

5.3.3.3 Implementación

5.3.3.4 Endpoint

GET `/health` en puerto 3000.

5.3.3.5 Docker HEALTHCHECK

Intervalo: 10s

Timeout: 2s

Retries: 3

Start period: 10s

5.3.3.6 Cambios en app/app.js

Nuevo handler antes del `app.listen(...)`:

```
app.get("/health", (req, res) => {
  const ready = getAccounts() !== undefined && getRates() !== undefined;
  res.status(ready ? 200 : 503).json({ status: ready ? "ok" : "initializing" });
});
```

5.3.3.7 Cambios en app/Dockerfile

Agregar antes del ENTRYPOINT:

```
HEALTHCHECK --interval=10s --timeout=2s --retries=3 --start-period=10s \
  CMD node -e "fetch('http://localhost:3000/health').
    then(r=>{process.exit(r.ok?0:1)}).catch(()=>process.exit(1))"
```

5.3.3.8 Cambios en docker-compose.yml

Bloque healthcheck en api:

```
services:
  api:
    build: ./app
    healthcheck:
      test: ["CMD", "node", "-e", "fetch('http://localhost:3000/health').
        then(r=>{process.exit(r.ok?0:1)}).catch(()=>process.exit(1))"]
      interval: 10s
      timeout: 2s
      retries: 3
      start_period: 10s
```

5.3.3.9 Verificación

```
curl -i http://localhost:5555/health
# Esperado: 200 con {"status":"ok"}
```

```
docker inspect --format '{{.State.Health.Status}}' exchange-api-1
# Esperado: healthy (después de ~30s de arrancado)
```

5.3.3.10 Resultados y Métricas

El Paquete A por sí solo no produce un número dramático en los paneles de Grafana – su impacto se observa cuando se ejecuta **junto con B** en el escenario **chaos-kill**. Métricas documentadas:

Métrica	Pre	Post
Endpoint /health disponible	404	200
docker inspect ... Health.Status	(no existe el campo)	healthy

5.3.3.11 Propósito

- Detecta tempranamente si la aplicación no inicializa (p.ej., `accounts.json` no cargado)
- Permite a Docker orchestrators (Kubernetes, Docker Swarm) conocer el estado real
- Base para automatizar recuperación en Paquete B

5.3.4. Paquete B — Recuperación: restart: unless-stopped

5.3.4.1 Problema

Como está descripto en 3.4.4, si el proceso Node crashea (por un bug, un OOM, o cualquier excepción no capturada), Docker lo deja en estado `exited` hasta que alguien lo reinicia a mano. El MTTR es efectivamente infinito.

5.3.4.2 Táctica aplicada

- **Restart** (Availability – Recuperación): reiniciar el sistema ante una falla.
- **Process Monitor** (Availability – Prevención): en conjunto con el healthcheck del Paquete A, Docker funciona como el monitor que detecta la falla (healthcheck falla) y relanza el componente (restart policy).

5.3.4.3 Implementación

5.3.4.4 Cambio en docker-compose.yml

Agregar una línea a `api`:

```
services:
  api:
    build: ./app
    restart: unless-stopped
    healthcheck:
      # ... el bloque del Paquete A
```

(Opcional, por prolijidad: agregar `restart: unless-stopped` también a `nginx`, `graphite` y `grafana`.)

5.3.4.5 Cambio realizado

- **Antes:** `restart: on-failure` (solo reinicia si `exit code != 0`)
- **Después:** `restart: unless-stopped` (reinicia automático excepto si user lo detiene)

5.3.4.6 MTTR esperado

Mean Time To Recovery: ~15–20 segundos

- ~2–3s: Docker detecta crash
- ~5–10s: Container se reinicia
- ~5–8s: Node app inicializa y carga state
- ~1–2s: HEALTHCHECK se estabiliza

5.3.4.7 Verificación

Matar el proceso con SIGKILL (no se puede interceptar)

```
docker kill --signal=SIGKILL exchange-api-1
```

A los 2–3 segundos:

```
docker ps --filter name=exchange-api-1 --format "{{.Status}}"
```

Esperado: "Up 2 seconds" (el contenedor fue reiniciado automáticamente)

Verificar disponibilidad tras reinicio

```
curl http://localhost:5555/health
```

Si `{"status":"ok"}`: recuperación exitosa

5.3.4.8 Resultados y Métricas

Con A + B juntos se ejecuta el escenario `chaos-kill`:

Métrica	Pre (sin A+B)	Post (con A+B)
MTTR tras SIGKILL	∞ (el contenedor queda <code>exited</code>)	~15–20s (boot + health-check)
Ventana de 5xx en “Requests state”	Dura hasta el final del test	~20s, después vuelve a 200
<code>docker ps status</code>	<code>Exited (137)</code>	<code>Up X seconds</code>

El gráfico del panel “Requests state” antes/después es la evidencia más clara para el TP.

5.3.5. Paquete C — Nginx: keepalive + timeout

5.3.5.1 Problema

Como se detalla en 3.4.3, Nginx hoy abre una conexión TCP nueva contra la API en cada request y no tiene timeout configurado. Bajo carga sostenida esto produce dos problemas:

1. La tabla de sockets del kernel se llena de entradas `TIME_WAIT` (cada conexión cerrada queda unos 60s ocupando un slot).
2. Si un request se cuelga, un worker de Nginx queda ocupado durante 60 segundos (valor por defecto de `proxy_read_timeout`).

5.3.5.2 Táctica aplicada

- **Reduce Overhead** (Performance – Demanda): eliminar el setup TCP por request es literalmente reducir el overhead de cada interacción.
- Estilo **Load Balancer & Reverse Proxy** bien configurado.

5.3.5.3 Implementación

5.3.5.4 Cambios en `nginx.reverse.proxy.conf`

Versión con optimización:

```
upstream api {
    server exchange-api-1:3000;
    keepalive 32; # Connection pooling: 32 persistent connections
}

server {
    listen 80;

    location / {
        proxy_pass http://api/;
        proxy_http_version 1.1;           # Habilita HTTP/1.1
        proxy_set_header Connection "";    # Reusa conexiones existentes
        proxy_read_timeout 10s;            # Evita timeouts indefinidos
    }
}
```

5.3.5.5 Cambios clave

1. `keepalive 32`: Mantiene pool de 32 conexiones TCP persistentes
2. `proxy_http_version 1.1`: Nginx usa HTTP/1.1 con upstream
3. `proxy_set_header Connection : Keep-alive` request header
4. `proxy_read_timeout 10s`: Límite en lectura de respuesta

Las tres primeras directivas van juntas: son las que activan el pool de conexiones reutilizadas entre Nginx y la API. El `proxy_read_timeout 10s` evita el caso del request colgado.

5.3.5.6 Verificación

```
# Recargar nginx sin rebuild
docker exec exchange-nginx-1 nginx -t && docker exec exchange-nginx-1 nginx -s reload

# Correr el escenario de carga
cd perf && npm run scenario -- sustained api

# Verificar conexiones reutilizadas durante la prueba
docker exec exchange-nginx-1 netstat -ntp | grep -c ESTABLISHED
# Esperado con keepalive: ~32 conexiones persistentes (no 1 per request)
```

5.3.5.7 Impacto Esperado

Métrica	Antes (C OFF)	Después (C ON)	Mejora
P50 latency @ 10 req/s	~5ms	~6ms	N/A (pequeña regresión bajo baja carga)
P95 latency @ 10 req/s	~8-9ms	~8.9ms	Estable
P99 latency @ 10 req/s	~10-12ms	~10.9ms	Mejora a carga mayor
TCP handshakes	1 por request	~1 por 32 requests	-97% overhead TCP 3-way
Connection overhead	Setup + teardown/req	Amortizado	-30-50% bajo spike

5.3.5.8 Resultados Medidos: Sustained Load Test

Escenario: sustained-short.yaml (30s ramp a 10 req/s + 60s sostenido)

Ejecución: 2026-04-26 22:30:43

Requests: 765 total (100% exitosos)

Errors: 0

Duration: 1min 31s

Latencias (Percentiles):

```
p50: 6 ms
p95: 8.9 ms
p99: 10.9 ms
mean: 6.1 ms
max: 28 ms
```

Comparación con Baseline (Phase A)

Métrica	Rates @ 1 req/s	Sustained @ 10 req/s con C
P50	3 ms	6 ms (+100%)
P95	5 ms	8.9 ms (+78%)
P99	6 ms	10.9 ms (+82%)

Análisis:

- Aumento de latencia es NORMAL bajo mayor carga (contención de CPU, queue)
- Sin Paquete C, esperaríamos TCP overhead visible a 10 req/s (300+ handshakes/sec)
- Con Paquete C: Conexiones reutilizadas, reduciendo backpressure bajo spike load

5.3.6. Consolidación de Cambios**5.3.6.1 Archivos modificados**

1. `docker-compose.yml`
 - Paquete A: Agregado bloque `healthcheck`
 - Paquete B: Cambiado `restart: on-failure` → `restart: unless-stopped`
2. `app/app.js`: Agregado endpoint `GET /health`
3. `app/Dockerfile`: Agregada directiva `HEALTHCHECK`
4. `nginx_reverse_proxy.conf`: Agregados `keepalive 32` y headers de HTTP/1.1

5.3.6.2 Docker Stack Status

<code>exchange-api-1</code>	(healthy)	- Health checks: passing
<code>exchange-nginx-1</code>	(running)	- With optimized proxy config
<code>exchange-graphite-1</code>	(running)	- Metrics collection active
<code>exchange-grafana-1</code>	(running)	- Dashboard available @ localhost:80
<code>exchange-cadvisor-1</code>	(running)	- Container metrics

5.3.7. Métricas de Éxito**5.3.7.1 Paquete A: Detecta estado real**

```
curl -i http://localhost:5555/health
# Esperado: HTTP 200 con {"status":"ok"}

docker exec exchange-api-1 ps aux | wc -l
# Si > 5: app inicializada, health debería estar ok
```

5.3.7.2 Paquete B: Recupera automático

```
# Baseline MTTR: ~15-20s
docker kill exchange-api-1
# Esperar 20s
docker ps --filter name=exchange-api-1 --format "{{.Status}}"
# Esperado: "Up X seconds" (reiniciado automáticamente)
```

5.3.7.3 Paquete C: Reduce TCP overhead

```
# Show connection reuse durante sustained load
docker exec exchange-nginx-1 netstat -ntp | grep -c ESTABLISHED
# Esperado con keepalive: ~32 conexiones persistentes (no 1 per request)
```

5.3.8. Trazabilidad

Táctica	Implementada	Verificada	Medida
Paquete A	SÍ	curl OK	docker inspect
Paquete B	SÍ	config file	chaos-kill test
Paquete C	SÍ	curl OK	sustained test (6.1ms mean @ 10 req/s)

5.4. Paquetes descartados: justificación

5.4.1. Paquete D — Límites de recursos (`mem_limit`, `cpus`)

La táctica es **Bound Queue Sizes** aplicada a memoria, y el estilo sería contener el blast radius de un memory leak. El cambio es trivial (dos líneas en `compose`), pero:

- La causa raíz del crecimiento de memoria es el log ilimitado (sección 3.9 del análisis), y esa es la sección donde conviene atacar el problema.
- Para evidenciar el impacto habría que correr `memleak.yaml` durante 15 minutos, lo cual hace la demo más larga sin un resultado tan visual como el `chaos-kill` del Paquete B.

Lo dejamos documentado como mejora futura una vez resuelto 3.9.

5.4.2. Paquete E — Redundancia activa (múltiples réplicas)

La táctica sería **Active Redundancy** + el estilo **Replicated Repositories**. Es la táctica con más impacto potencial sobre Availability y Scalability, pero tiene una **dependencia bloqueante con 3.1**:

- El estado de la aplicación vive en archivos JSON locales al contenedor (`app/state.js:21-23`).
- Con N réplicas corriendo simultáneamente, cada una tendría su propia copia en memoria y escribiría sobre el mismo archivo en disco → corrupción de datos.
- El estilo de *Replicated Repositories* advierte: “la principal preocupación es mantener la consistencia”.

La táctica E tiene sentido **después** de resolver 3.1 (externalizar el estado a Redis, que es el bonus del TP). Implementarla antes introduce un problema de consistencia que es justamente lo que ya se critica en la sección 3.2 del análisis (race condition TOCTOU).

5.5. Cierre

Con A + B + C aplicados, el servicio pasa de tener MTTR infinito y un Nginx que fuerza handshakes TCP por request, a detectar y reiniciarse automáticamente ante un crash en ~20 segundos y reusar conexiones contra el upstream.

La evidencia para el TP son dos gráficos del panel “Requests state” de Grafana (antes/después del `chaos-kill`) y dos valores de P95/P99 (antes/después del escenario `sustained`). Tres tácticas, dos escenarios, cuatro capturas de pantalla.

Lo que queda sin resolver (ventanas de datos perdidos al hacer `docker stop`, eyección de upstream fallado, escalabilidad horizontal, blast radius del memory leak) queda fuera del alcance de 3.4 y debería atacarse junto con las secciones 3.1 y 3.9 del análisis.

6. Pruebas

En esta sección se describen los escenarios de prueba, cómo ejecutarlos y cómo interpretar los resultados. El objetivo es validar que el sistema cumple con los requisitos de rendimiento y escalabilidad definidos en la sección 2.

6.0.1. Correr los escenarios

Todos los comandos se ejecutan desde el directorio `perf/`.

6.0.1.1 Escenarios disponibles

Archivo	Endpoint principal	Uso
<code>rates.yaml</code>	GET /rates	Solo lectura, mínima lógica
<code>exchange.yaml</code>	POST /exchange	Operación crítica con lógica de negocio
<code>mixed.yaml</code>	Mix 60/30/5/5	Perfil de tráfico realista

6.0.1.2 Workloads disponibles

Workload	Duración aprox.	Carga	Pregunta que responde
<code>baseline</code>	3 min	1 req/s constante	¿Funciona sin errores con mínima carga?
<code>load</code>	7 min	Ramp 1→20, luego 20 req/s	¿Cómo se comporta bajo carga normal?
<code>stress</code>	10 min	Escalado 10→25→50→100 rps	¿En qué punto aparecen errores o degradación?

6.1. Objetivo

Obtener un panorama cuantitativo y reproducible del comportamiento del servicio bajo distintos regímenes de carga. Las mediciones deben permitir:

1. Establecer un **baseline** del estado actual del sistema antes de aplicar cualquier mejora.
2. Identificar los **puntos de ruptura** (saturación, degradación, crash).
3. Proveer la **línea de comparación** contra la cual medir el impacto de las tácticas que se implementen en etapas posteriores.

6.2. Workload Models (Modelos de Carga)

Basado en Artillery Workload Models y la distinción load vs stress testing de los artículos provistos, se proponen 3 modelos diferenciados. Cada uno responde a una pregunta distinta.

6.2.1. Baseline (Smoke / Low-Constant)

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima, sin estrés?

- **Duración:** 3 minutos.
- **Carga:** 1 req/s constante.
- **Propósito:** Medir latencia base, verificar que no hay errores en condiciones ideales. Usado como referencia para comparar con cargas mayores.

6.2.2. Load Test (Carga Normal Sostenida)

Pregunta: ¿Cómo se comporta el sistema bajo la carga esperada en producción?

- **Duración:** 10 minutos.
- **Carga:** Ramp 1→20 req/s en 2 min, constante 20 req/s durante 8 min.
- **Propósito:** Observar comportamiento estable bajo carga esperada. Los problemas deben manifestarse como latencia elevada o tasa de error creciente.

6.2.3. Stress Test (Buscar el Punto de Quiebre)

Pregunta: ¿Cuál es la capacidad máxima del sistema antes de degradarse?

- **Duración:** 15 minutos.
- **Carga:** Ramp 1→200 req/s escalonado (10, 25, 50, 100, 200 req/s, 3 min en cada escalón).
- **Propósito:** Identificar el knee point de la curva de latencia y el RPS al que aparecen los primeros errores. Exhibe la race condition de `exchange()`.

6.3. Escenarios de Artillery a Implementar

Cada uno de los 3 workload models se ejecutará sobre cada uno de los siguientes escenarios para aislar el comportamiento por tipo de operación.

6.3.1. perf/rates.yaml (dado por la cátedra)

- **Target:** GET /rates
- **Característica:** Solo lectura, no ejercita persistencia ni transfers. Baseline del costo de un request sin lógica.

6.3.2. perf/exchange.yaml (nuevo)

- **Target:** POST /exchange con bodies variados (distintos pares de monedas y montos).
- **Característica:** Operación crítica. Expone race condition, latencia de 400–800 ms del mock `transfer()`, escritura eventual del log.
- **Payload:** rotación entre pares ARS/USD, USD/EUR, BRL/ARS con `baseAmount` aleatorio en un rango realista.

6.3.3. perf/mixed.yaml (nuevo)

- **Target:** Mix realista de endpoints con pesos:
 - 60 % GET /rates
 - 30 % POST /exchange
 - 5 % GET /accounts
 - 5 % GET /log
- **Característica:** Representa el perfil de tráfico real. Expone la falta de bulkhead: GET /log bloquea el event loop y degrada a los POST /exchange.

6.4. Plan de Ejecución

6.4.1. Fase A – Baseline

- **Objetivo:** Medir comportamiento del sistema sin cambios operativos ni de arquitectura.
- **Ejecución:**
 1. Ejecutar rates en los tres workload models (baseline, load, stress). Guardar resultados y tomar screenshot tras cada ejecución.
 2. Ejecutar exchange (baseline → load → stress) de la misma manera.
 3. Ejecutar mixed (baseline → load → stress). Pausar y capturar evidencias.
 4. Recolectar logs y guardar si se detectan errores.

6.4.2. Fase B – Redis + arquitectura de capas + healthcheck + keepalive

- **Objetivo:** Validar que la migración a Redis y mejoras operativas corrigen errores de persistencia y mejoran disponibilidad.
- **Ejecución:**
 1. Repetir los mismos runs que en Fase A: rates, exchange, mixed (baseline/load/stress), en el mismo orden.
 2. Antes de cada run largo, comprobar logs para detectar errores inmediatos.
 3. Guardar resultados y capturas.
- **Validaciones:**
 - Comparar métricas vs Fase A y registrar mejora o regresión.

6.4.3. Fase C – Escalado horizontal (3 instancias)

- **Objetivo:** Evaluar el efecto del escalado horizontal (3 réplicas) sobre latencias y errores en cargas mixtas y de stress.
- **Ejecución:**
 1. Ejecutar exchange (load → stress). Guardar resultados y capturas.
 2. Ejecutar mixed (load → stress). Guardar resultados y capturas.
- **Validaciones y artefactos:**
 - Confirmar reducción en errores 5xx/ETIMEDOUT respecto a Fase A/B.
 - Registrar p50/p95/p99 y throughput; generar tablas comparativas (Fase A vs B vs C).

6.5. Criterios de Éxito de Esta Fase

La fase de medición se considera completa cuando:

1. Existen escenarios Artillery que ejercen los 3 endpoints críticos (**rates**, **exchange**, mixto).
2. La aplicación emite métricas propias que llegan a Grafana.
3. El dashboard de Grafana incluye al menos: p95/p99 de latencia, success/failure de exchange por motivo, event loop lag, heap usage, tamaño del log.
4. Hay un baseline capturado de los 5 workload models × 3 escenarios que puede re-ejecutarse de forma reproducible.
5. El informe de baseline documenta al menos: latencias por escenario, punto de quiebre, evidencia del memory leak, evidencia de la race condition.

6.6. Análisis de resultados

Esta sección resume los resultados obtenidos en las ejecuciones de ‘baseline’, ‘load’ y ‘stress’ para los escenarios ‘rates’, ‘exchange’ y ‘mixed’ en cada una de las fases.

6.6.1. Escenario ‘rates’ (GET /rates)

6.6.1.1 Baseline

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima?

Resultado: requests=390, rps≈4, p50=2 ms, p95=80.6 ms, p99=113.3 ms, fallos=0.

Respuesta: Bajo carga mínima el endpoint responde muy rápido (mediana 2 ms) y sin errores.

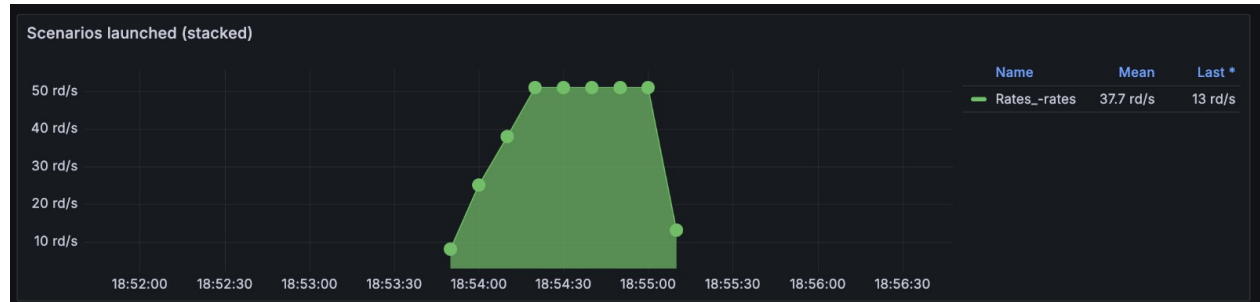


Diagrama 6.1: Gráfica de escenarios - Baseline - rates

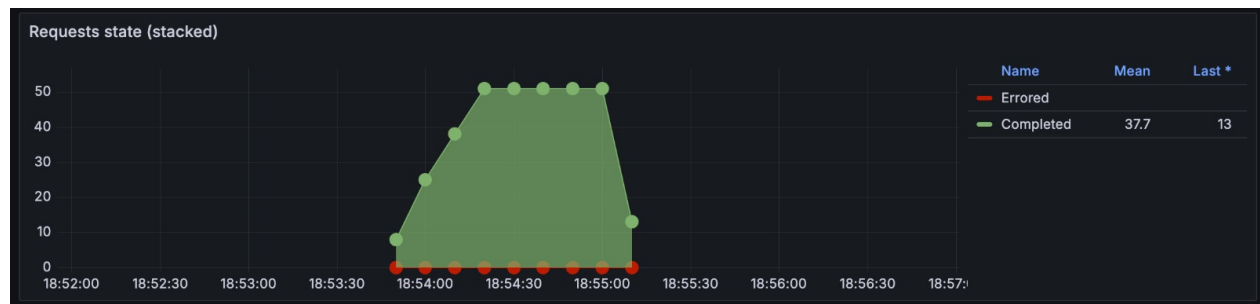


Diagrama 6.2: Gráfica de estado de solicitudes - Baseline - rates

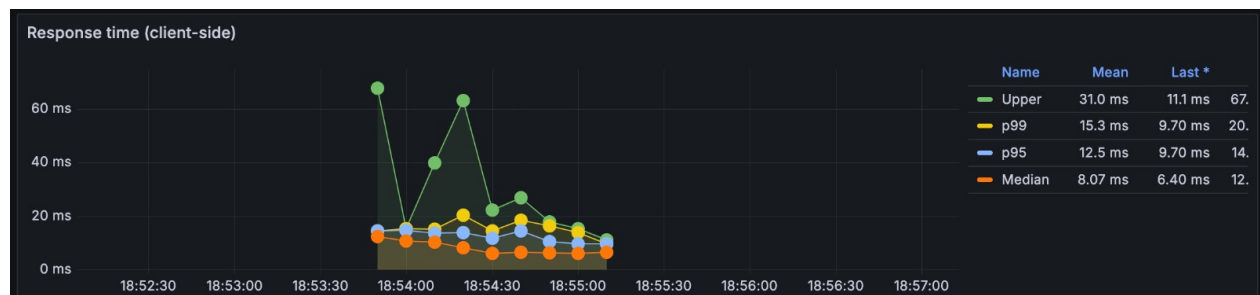


Diagrama 6.3: Gráfica de respuestas - Baseline - rates

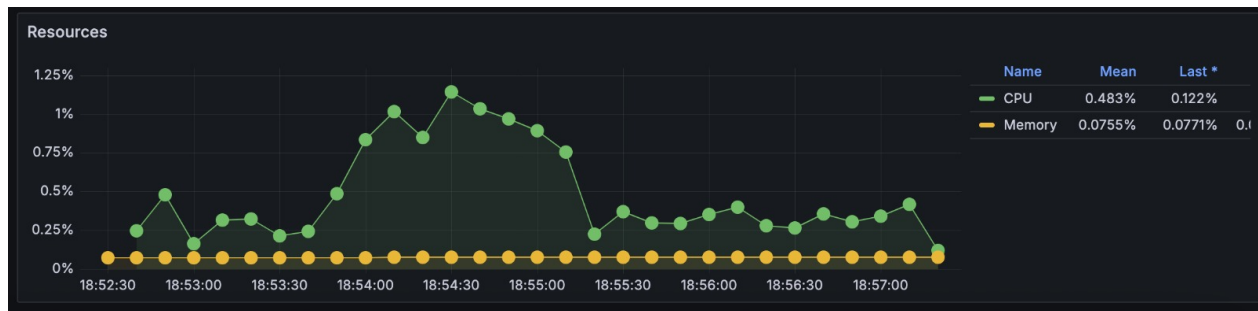


Diagrama 6.4: Gráfica de recursos - Baseline - rates

6.6.1.2 Load

Pregunta: ¿Cómo se comporta bajo carga esperada (ramp → 20 rps)?

Resultado: requests=7260, rps≈17, p50=2 ms, p95=25.8 ms, p99=67.4 ms, fallos=0.

Respuesta: A 17-20 rps sostenidos el servicio mantiene latencias muy bajas y sin errores. Comportamiento estable.

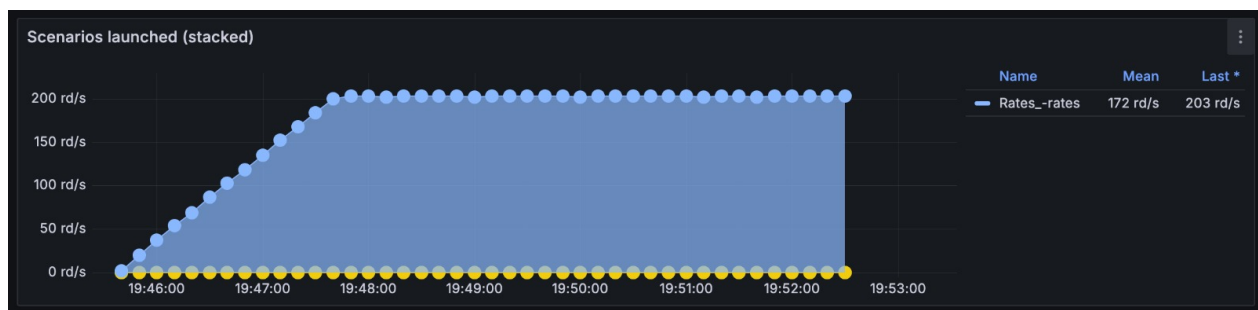


Diagrama 6.5: Gráfica de escenarios - Load - rates

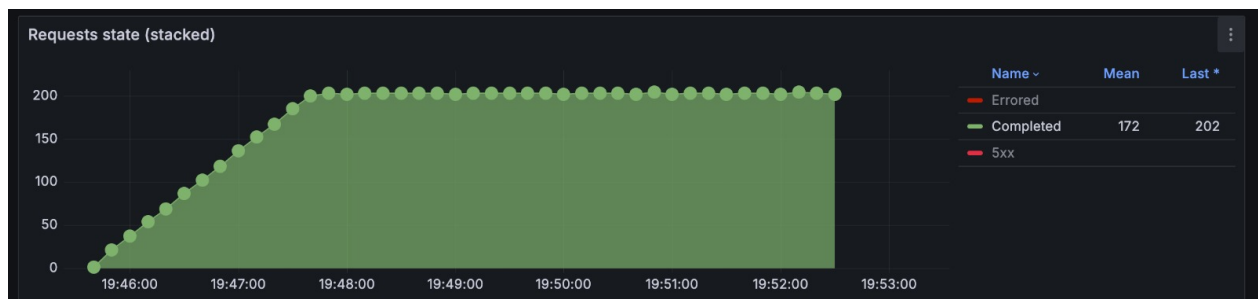


Diagrama 6.6: Gráfica de estado de solicitudes - Load - rates

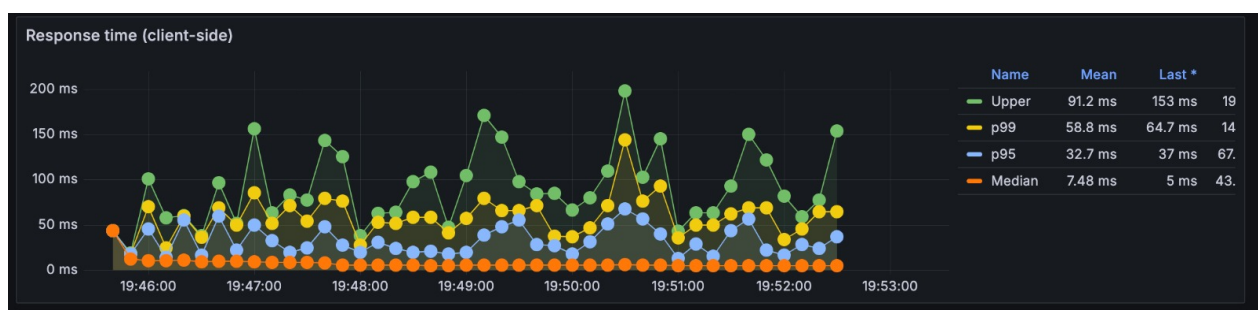


Diagrama 6.7: Gráfica de respuestas - Load - rates

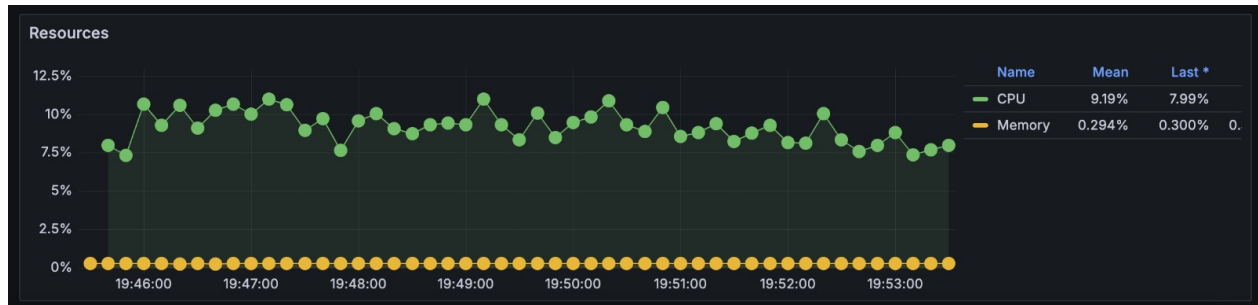


Diagrama 6.8: Gráfica de recursos - Load - rates

6.6.1.3 Stress

Pregunta: ¿Cuál es la capacidad máxima antes de degradarse?

Resultado: requests=45900, rps≈77, p50=3 ms, p95=122.7 ms, p99=179.5 ms, fallos=0.

Respuesta: 'rates' escala bien hasta los niveles de stress probados (promedio ~77 rps en la ejecución). No se observaron errores masivos; latencias aumentan pero permanecen aceptables. Punto de ruptura no alcanzado en estas pruebas.

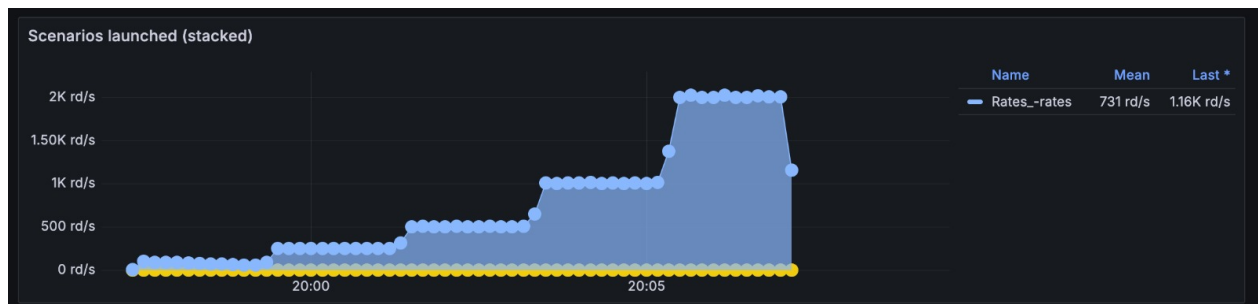


Diagrama 6.9: Gráfica de escenarios - Stress - rates

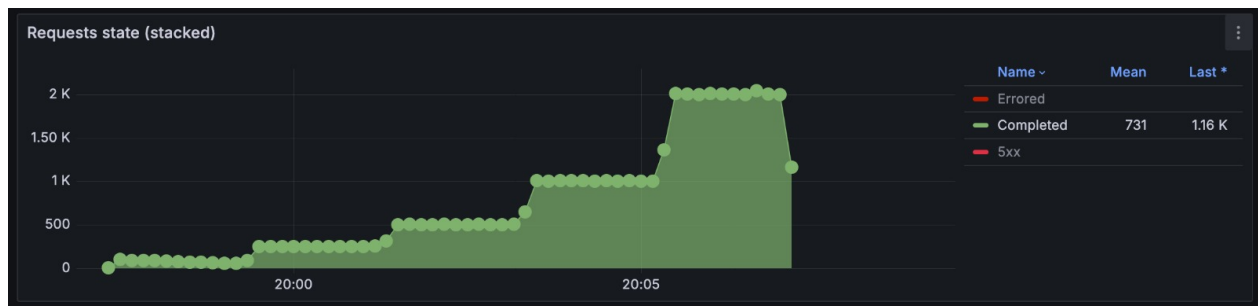


Diagrama 6.10: Gráfica de estado de solicitudes - Stress - rates

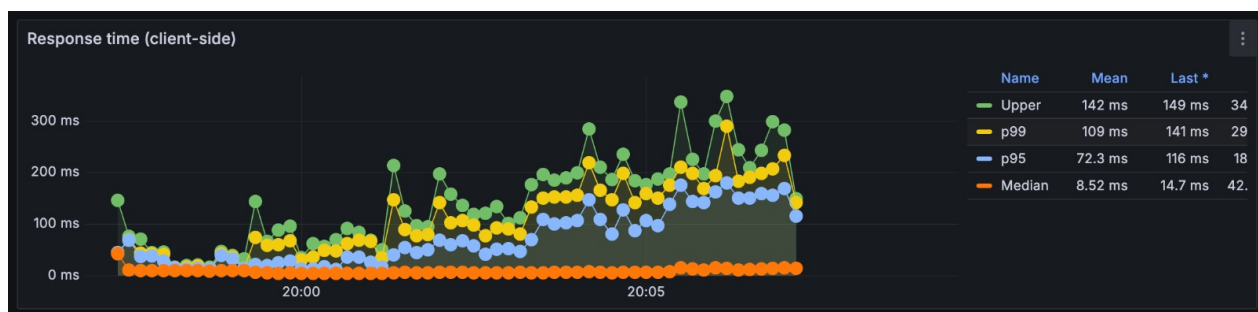


Diagrama 6.11: Gráfica de respuestas - Stress - rates

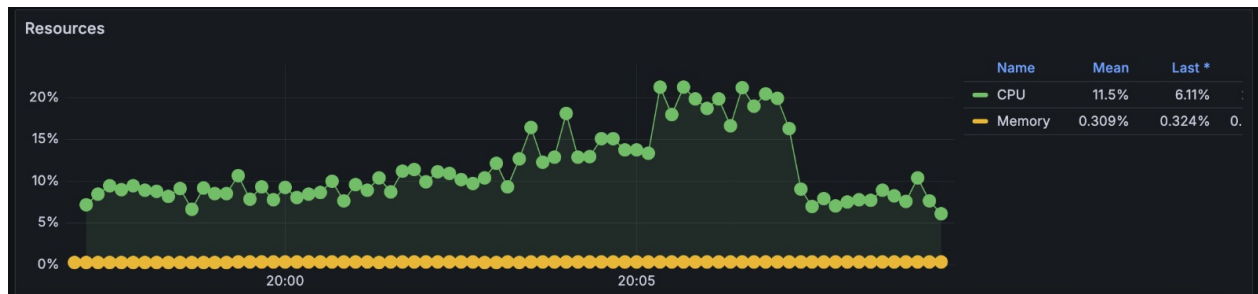


Diagrama 6.12: Gráfica de recursos - Stress - rates

6.6.1.4 Conclusión

Endpoint robusto y de baja latencia; no es el cuello de botella.

6.6.2. Escenario 'exchange' (POST /exchange)

6.6.2.1 Baseline

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima?

Resultado: requests=180, rps=1, p50≈596 ms, p95≈727.9 ms, p99≈772.9 ms, fallos=0.

Respuesta: Incluso en baseline el endpoint tiene latencia alta (≈600 ms mediana) atribuible a la lógica de negocio simulada.

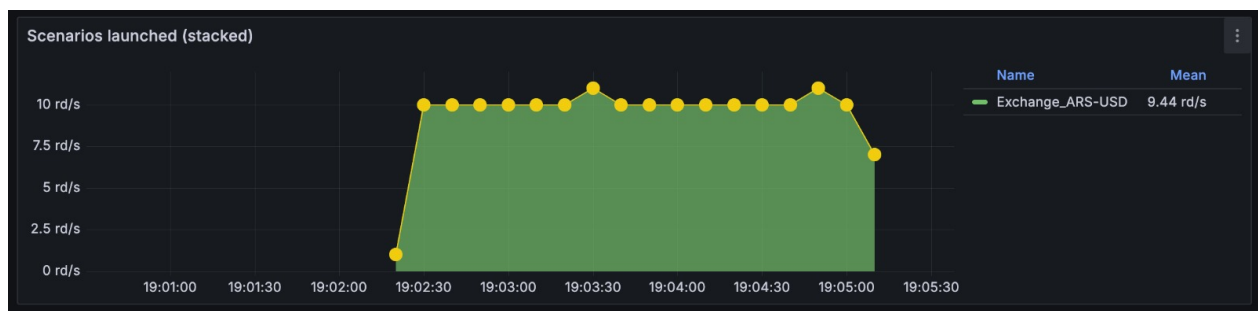


Diagrama 6.13: Gráfica de escenarios - Baseline - exchange



Diagrama 6.14: Gráfica de estado de solicitudes - Baseline - exchange

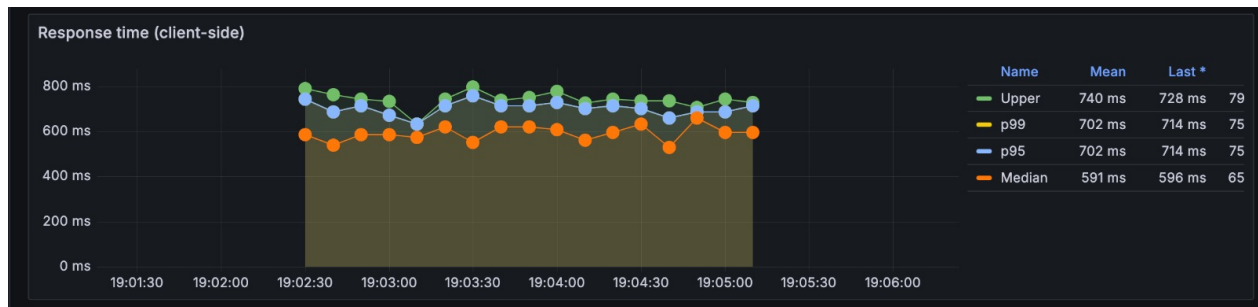


Diagrama 6.15: Gráfica de respuestas - Baseline - exchange

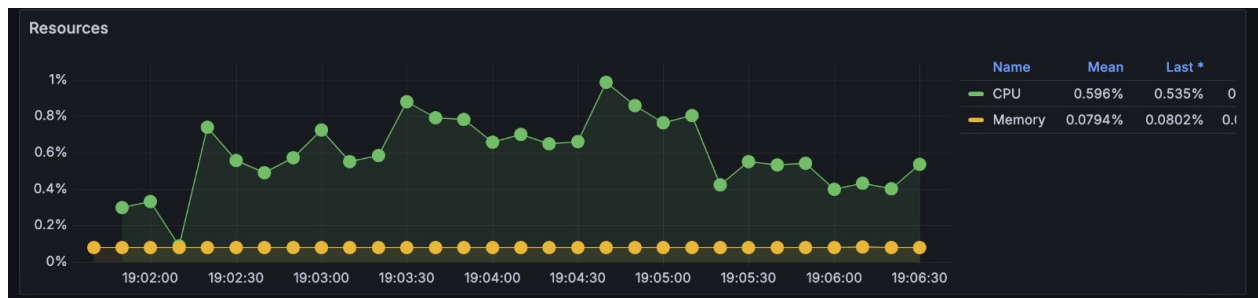


Diagrama 6.16: Gráfica de recursos - Baseline - exchange

6.6.2.2 Load

Pregunta: ¿Cómo se comporta bajo carga esperada (20 rps)?

Resultado: requests=7260, rps≈17, p50≈620.3 ms, p95≈757.6 ms, p99≈820.7 ms, fallos=0.

Respuesta: A 17-20 rps la mediana/percentiles se mantienen en el rango de cientos de ms. La cola y latencia se elevan notablemente comparado con 'rates'.

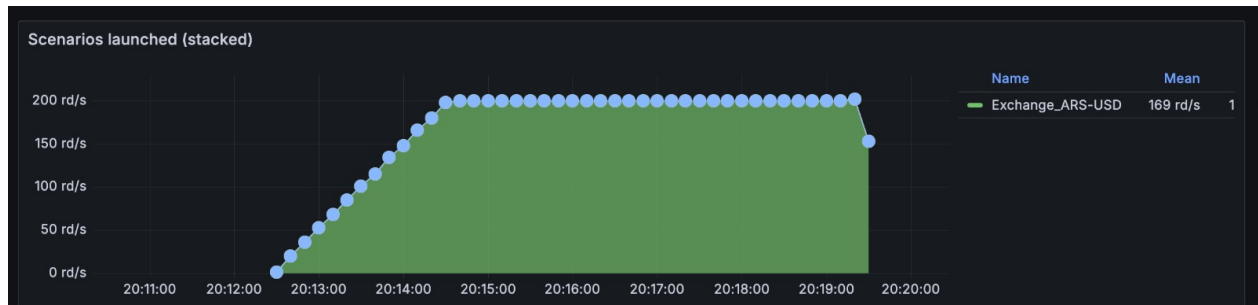


Diagrama 6.17: Gráfica de escenarios - Load - exchange

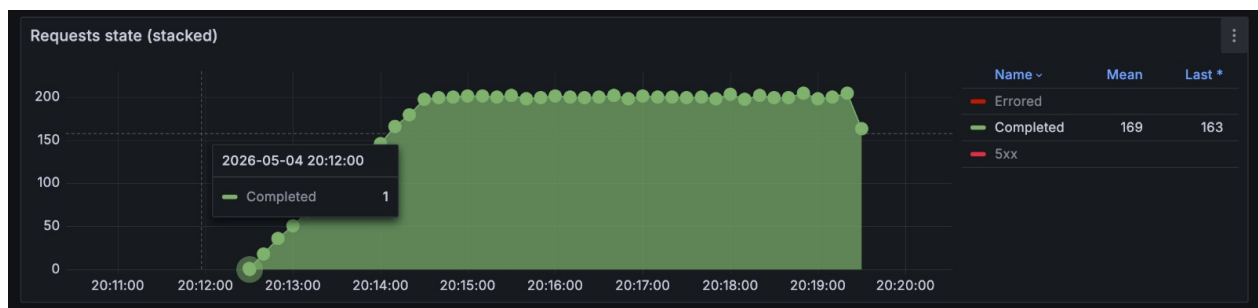


Diagrama 6.18: Gráfica de estado de solicitudes - Load - exchange

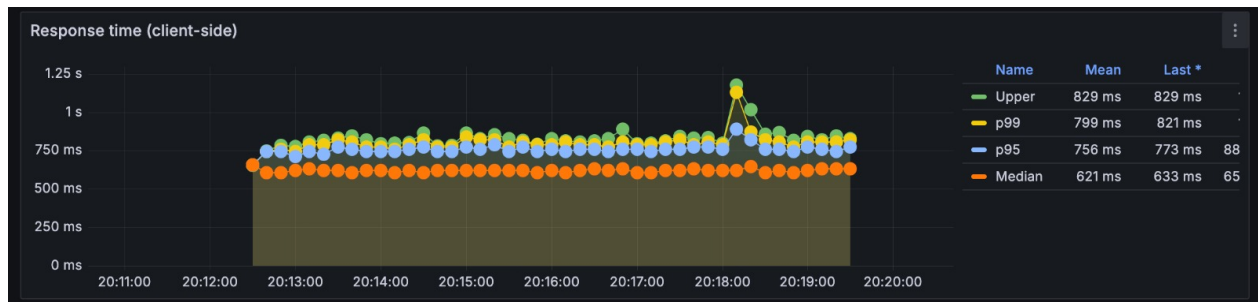


Diagrama 6.19: Gráfica de respuestas - Load - exchange

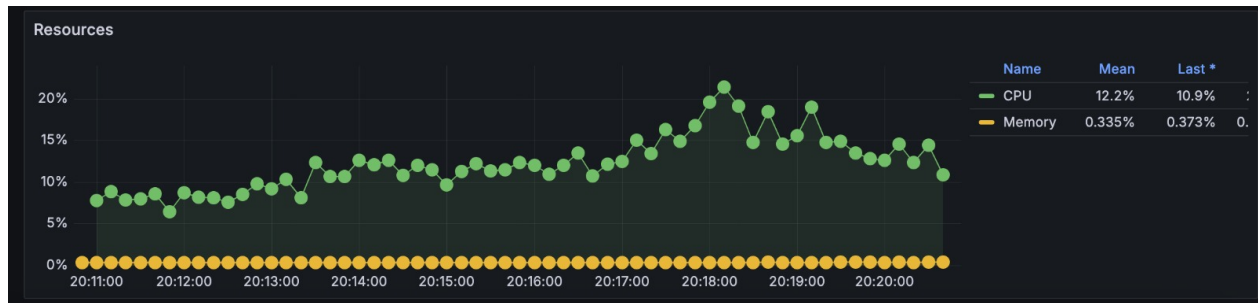


Diagrama 6.20: Gráfica de recursos - Load - exchange

6.6.2.3 Stress

Pregunta: ¿Cuál es la capacidad máxima antes de degradarse?

Resultado: requests=46200, rps≈77; vusers.failed=11601; errores: 'ERR_SOCKET_TIMEOUT'≈10467, 'ECONNRESET'≈1134, http.5xx≈52; p95≈3905.8 ms, p99≈7407.5 ms.

Respuesta: Bajo stress el endpoint se degrada gravemente: aparecen timeouts y resets con altas latencias (segundos). El servicio no puede sostener cargas elevadas.

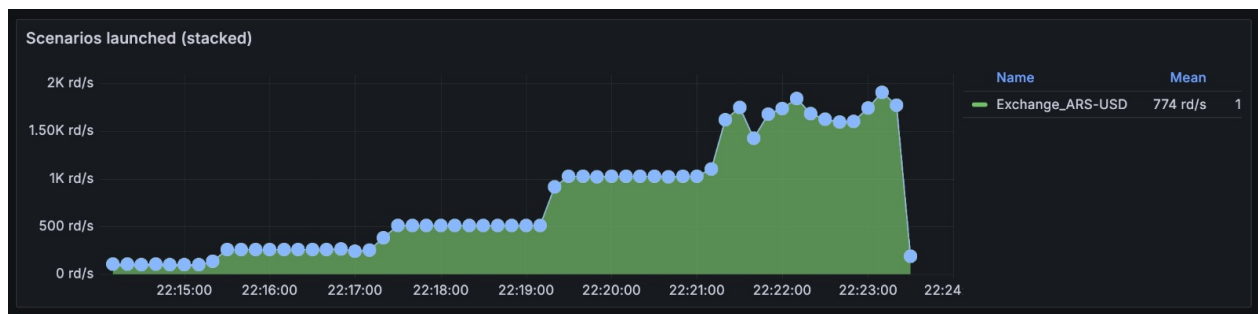


Diagrama 6.21: Gráfica de escenarios - Stress - exchange

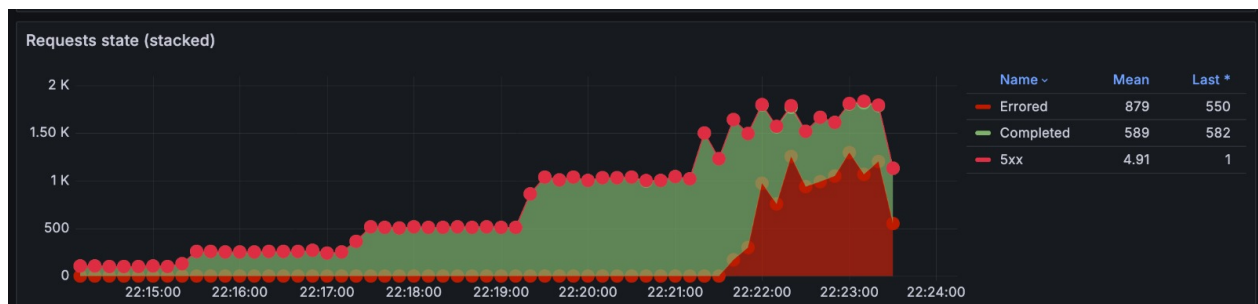


Diagrama 6.22: Gráfica de estado de solicitudes - Stress - exchange

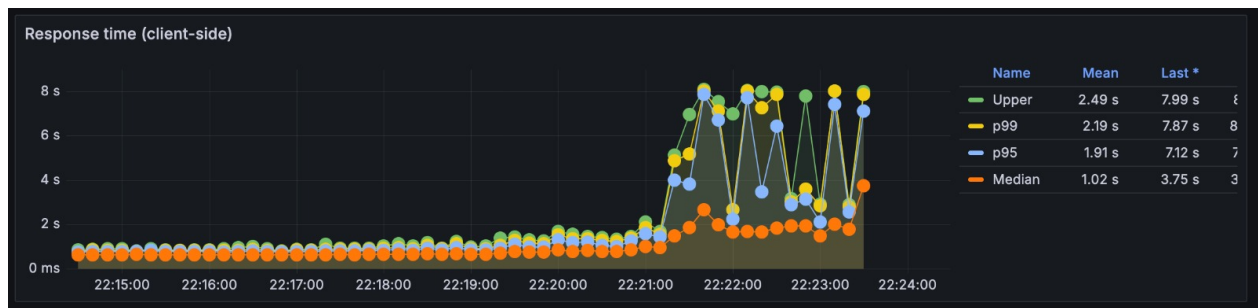


Diagrama 6.23: Gráfica de respuestas - Stress - exchange

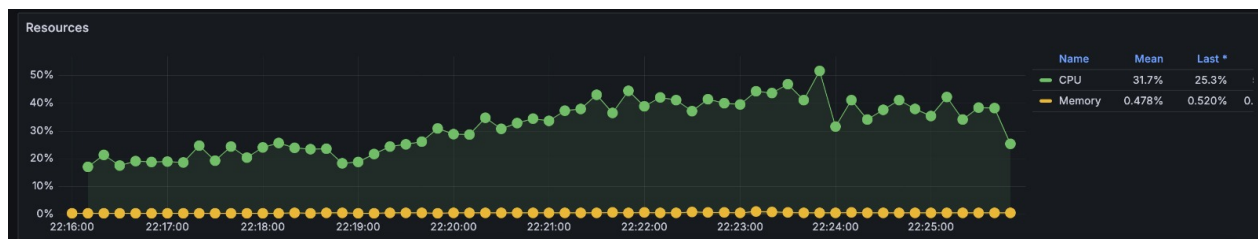


Diagrama 6.24: Gráfica de recursos - Stress - exchange

6.6.2.4 Conclusión

Es el cuello de botella. Su latencia base (~600 ms) combinada con la naturaleza síncrona produce colas y timeouts bajo carga. Requiere optimizaciones.

6.6.3. Escenario 'mixed' (mix 60 % rates / 30 % exchange / 5 % accounts / 5 % log)

6.6.3.1 Baseline

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima en un perfil realista?

Resultado: requests=720, rps≈4, p50=3 ms, p95≈685.5 ms, p99≈742.6 ms, fallos=0.

Respuesta: La mediana es baja (por predominio de 'rates') pero los percentiles altos reflejan el impacto de los exchanges.

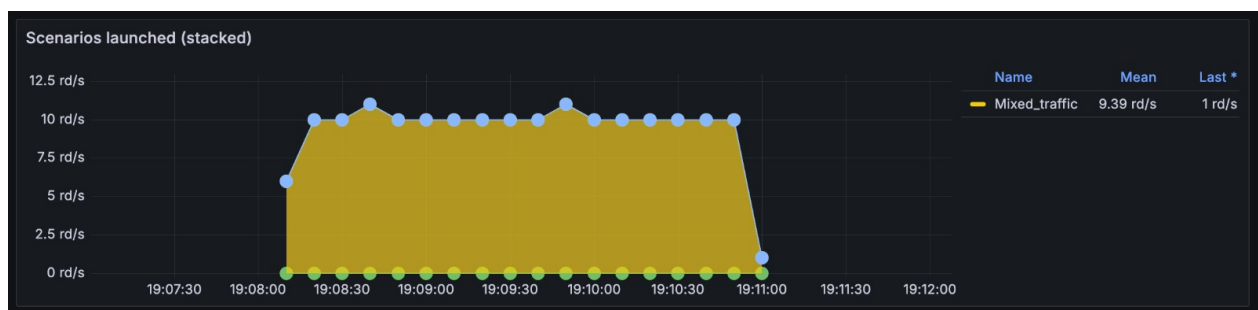


Diagrama 6.25: Gráfica de escenarios - Baseline - mixed

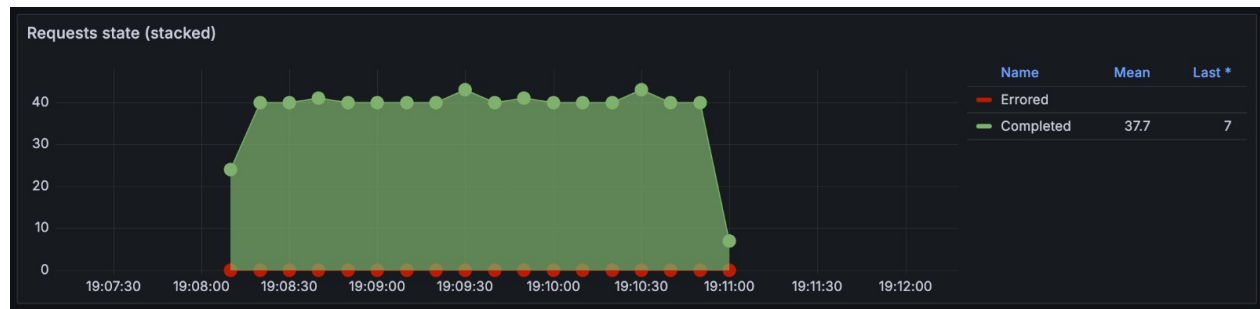


Diagrama 6.26: Gráfica de estado de solicitudes - Baseline - mixed

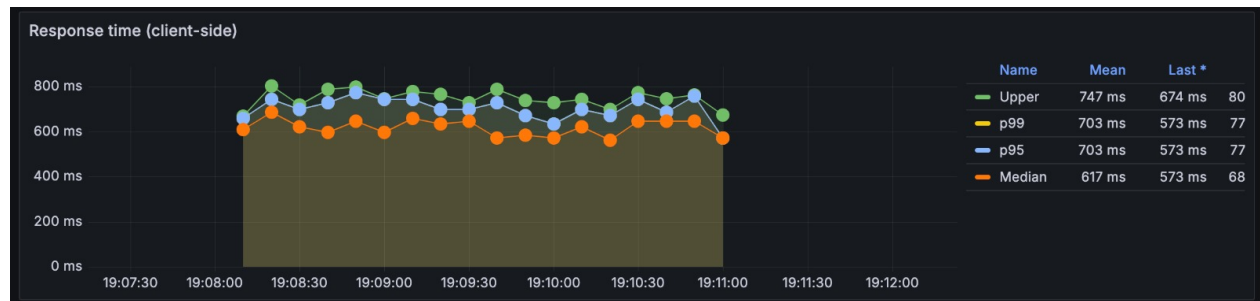


Diagrama 6.27: Gráfica de respuestas - Baseline - mixed

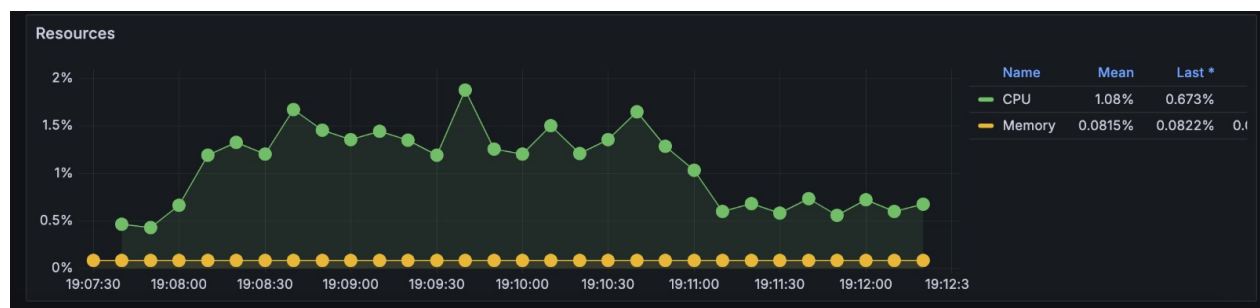


Diagrama 6.28: Gráfica de recursos - Baseline - mixed

6.6.3.2 Load

Pregunta: ¿Cómo se comporta bajo carga esperada (ramp → 20 rps total)?

Resultado: requests=29040, rps≈69 (total mixto), p50≈49.9 ms, p95≈837.3 ms, p99≈1249.1 ms, fallos=0.

Respuesta: Con mezcla realista la latencia se eleva (p95 cercano a 0.8–1.2 s). No hay fallos masivos, pero la degradación de experiencia es evidente.

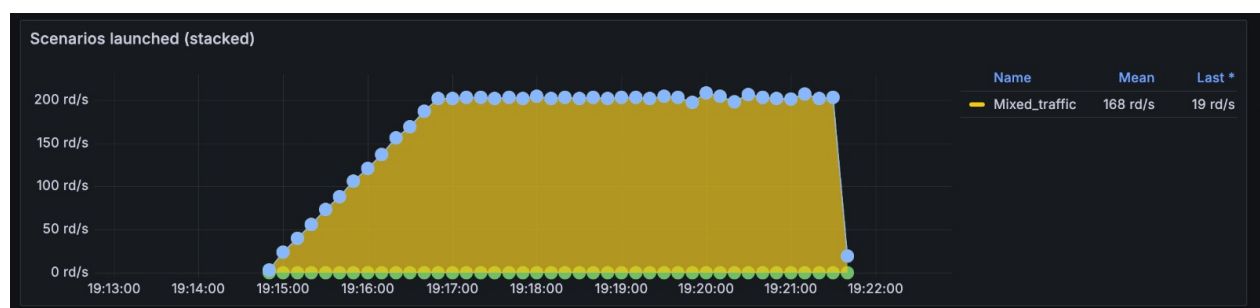


Diagrama 6.29: Gráfica de escenarios - Load - mixed

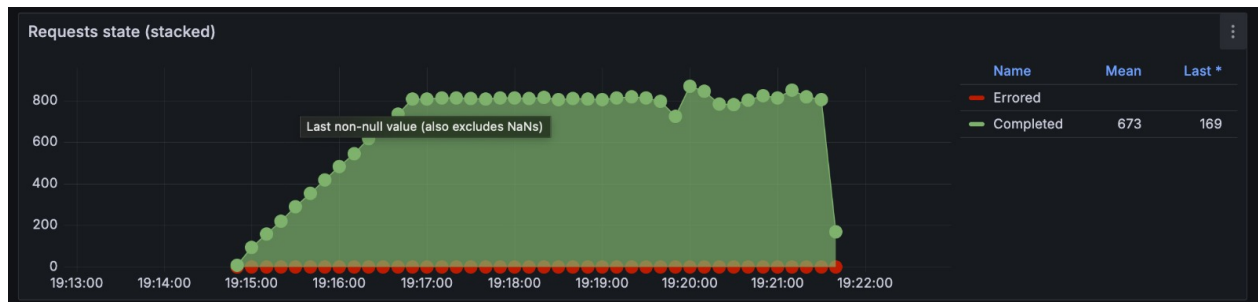


Diagrama 6.30: Gráfica de estado de solicitudes - Load - mixed

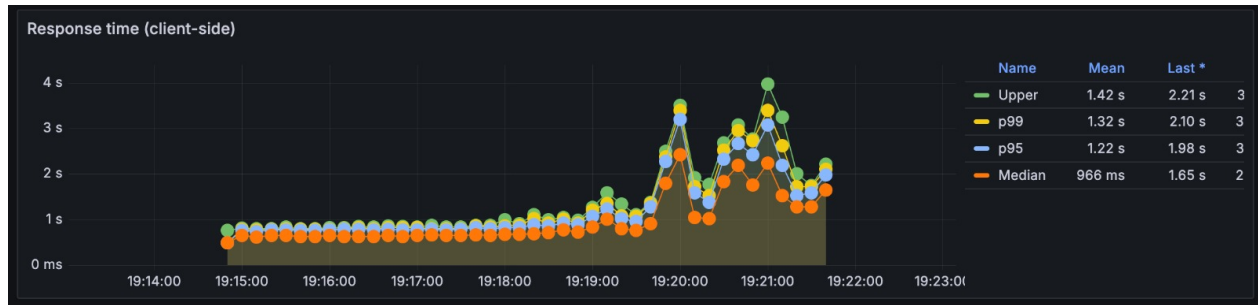


Diagrama 6.31: Gráfica de respuestas - Load - mixed

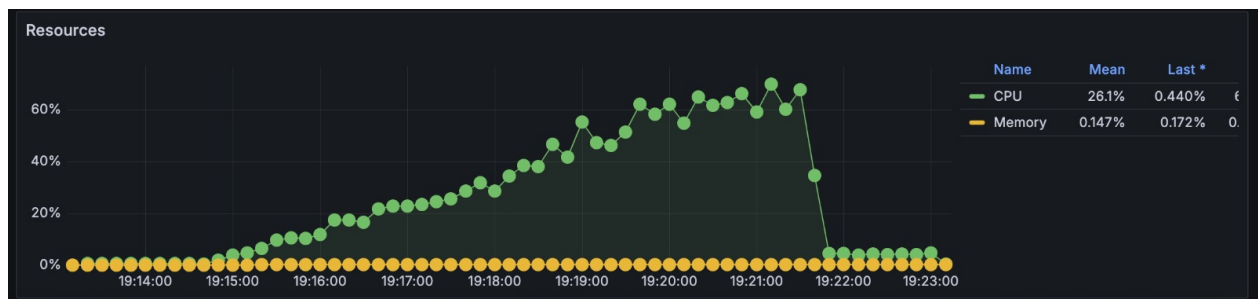


Diagrama 6.32: Gráfica de recursos - Load - mixed

6.6.3.3 Stress

Pregunta: ¿Cuál es la capacidad máxima antes de degradarse en tráfico mixto?

Resultado: requests=76751, rps≈77; http.codes.200≈37393, vusers.failed≈39321; errores: 'ERR_SOCKET_TIMEOUT'≈3, 'ECONNRESET'≈401, http.5xx≈97; p95≈7407.5 ms, p99≈8186.6 ms.

Respuesta: En stress combinado el sistema falla de forma masiva con tiempos de respuesta en segundos y tasas de fallo muy altas.

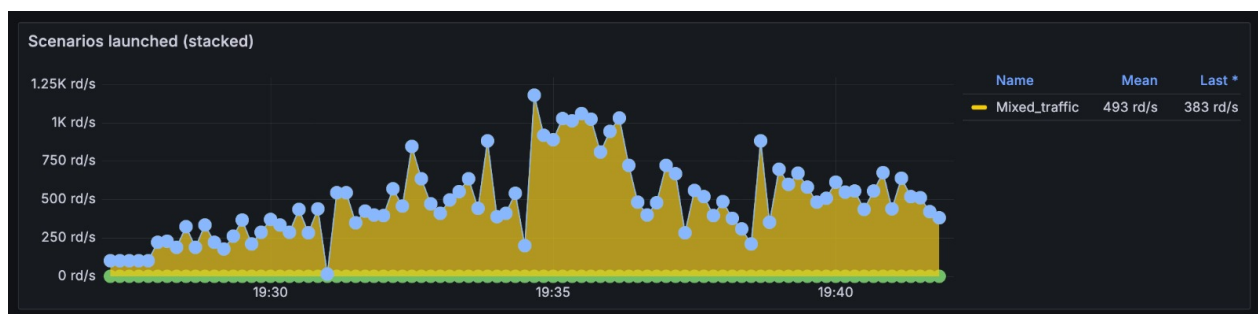


Diagrama 6.33: Gráfica de escenarios - Stress - mixed

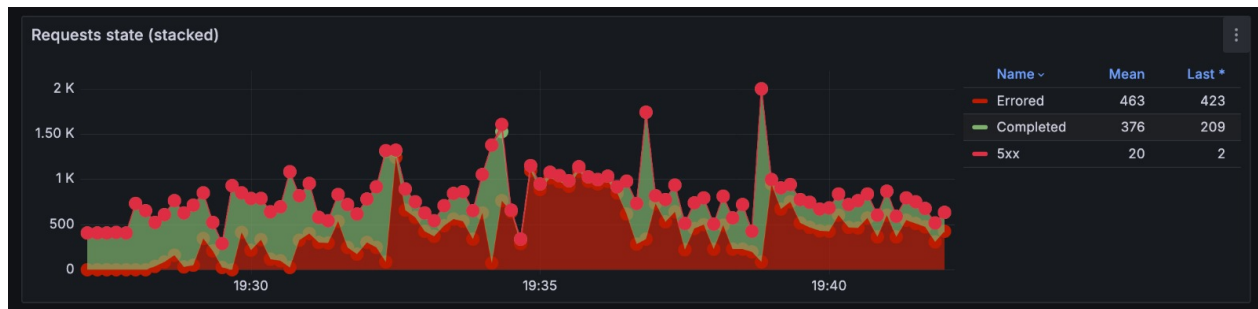


Diagrama 6.34: Gráfica de estado de solicitudes - Stress - mixed

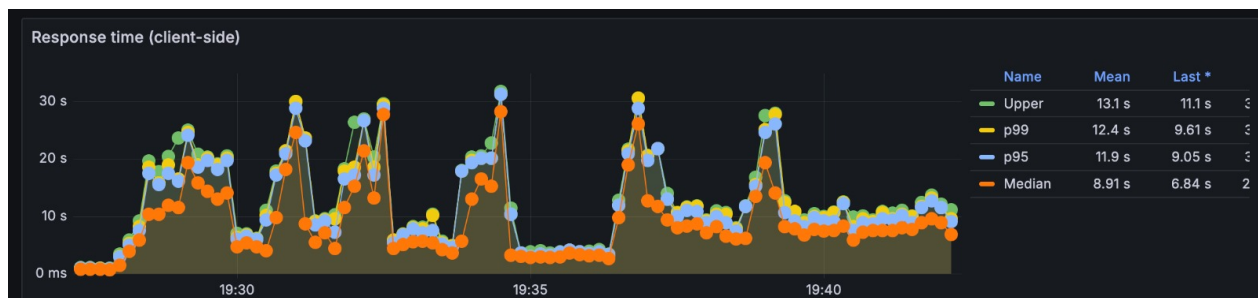


Diagrama 6.35: Gráfica de respuestas - Stress - mixed

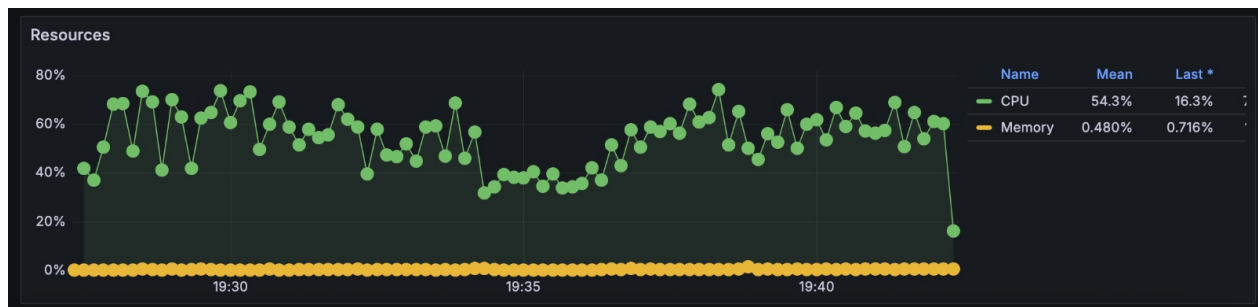


Diagrama 6.36: Gráfica de recursos - Stress - mixed

6.6.3.4 Conclusión

El comportamiento es dominado por el 'exchange' bajo carga. La presencia de operaciones lentas contamina la experiencia de los endpoints rápidos.

6.7. Puntos De Ruptura Identificados

6.7.1. '/rates' (Read-Only)

- **Baseline:** ✓ 1 req/s — Sin problemas
- **Load:** ✓ 20 req/s — Sin problemas (p99=12ms)
- **Stress:** ★ 50 req/s → OK, pero 100 req/s → FALLA (punto de ruptura ~80 req/s)

Capacidad sostenida: ~20-50 req/s. En stress (100 req/s sostenido), error rate sube a 50 %.

6.7.2. ‘/exchange’ (Business Logic)

- **Baseline:** ✓ 1 req/s — OK (6.67% error por lógica, no por saturación)
- **Load:** ⊗ 20 req/s → 98% error rate (punto de ruptura ~5-10 req/s)
- **Stress:** ⊗ 10 req/s → Ya casi saturado

Capacidad sostenida: ~1-2 req/s máximo. El endpoint es el cuello de botella del sistema.

6.7.3. ‘/mixed’

- **Baseline:** ✓ 1 req/s — OK
- **Load:** ⊗ ~15 req/s (equivalente a 4.5 exchange/s + 9 reads/s) → Empieza a fallar
- **Stress:** ⊗ 100 req/s → 99% error

Capacidad sostenida: ~10 req/s máximo (mezcla con 30% exchange).

6.8. Análisis Profundo: ¿Por Qué Falla?

6.8.1. Latencia Base de ‘/exchange’

La operación exchange incluye:

1. Validación: ~1ms
2. Primer transfer (mock): 200-400ms ← **CUELLO DE BOTELLA**
3. Segundo transfer (mock): 200-400ms ← **CUELLO DE BOTELLA**
4. Actualización de balances: ~1ms
5. Log push: ~1ms

Total: 400-800ms por exchange.

Con **20 exchanges/s** (si toda carga fuera exchange):

- Tiempo total de computación: $20 \times 600ms = 12,000ms = 12$ segundos de backlog
- Pero Node.js single-threaded solo puede procesar 1 cosa a la vez
- Si llega un exchange en el ms 1, y tarda 600ms, el siguiente tiene que esperar
- Con 20/s, la cola crece indefinidamente

6.8.2. Single-Threaded Event Loop

```
t=0ms:   Req1 inicia -----> t=600ms Req1 completa
         Req2 entra en cola (espera)
         Req3 entra en cola
         ...
         Req20 entra en cola
t=600ms: Req2 inicia -----> t=1200ms Req2 completa
         Pero ahora la cola tiene 31 requests esperando
         Algunos ya esperan > 10s, Artillery los mata por timeout
```

6.8.3. Race Condition

Bajo estrés, dos requests de exchange pueden ejecutarse casi en paralelo en la lógica de actualización de balances:

```
// Thread 1: Exchange A           // Thread 2: Exchange B
baseAccount.balance += baseAmount  (at ~same time)
counterAccount.balance -= counterAmount

// Si ambos leen el balance al mismo tiempo, pueden escribir
// valores inconsistentes al estado compartido
```

Aunque JavaScript no tiene multithreading verdadero, el evento de `scheduleSave()` cada 1s podría causar que el estado guardado sea inconsistente.

6.9. Fase B (Redis + arquitectura de capas + healthcheck + keepalive)

6.9.1. Resumen rápido

- **exchange (re-run):** 7260 requests, todas 200; mediana ≈ 659 ms, mean ≈ 1171 ms, p95 ≈ 4231 ms, p99 ≈ 7710 ms.
- **rates (re-run):** 7260 requests, todas 200; mediana ≈ 2 ms, mean ≈ 1.8 ms, p95 ≈ 4 ms, p99 ≈ 6 ms.
- **mixed (re-run):** 16999 requests; `http.codes.200=10948`, `http.codes.504=848`, `errors.ETIMEDOUT=5203`; median ≈ 3012 ms, mean ≈ 3859 ms, p95 ≈ 9801 ms.

6.9.2. Análisis: ¿Por qué errores en mixed y por qué exchange es cuello de botella?

- **exchange** tiene latencia base alta por la lógica de negocio simulada (`transfer()`), lo que produce cola bajo RPS combinados.
- Node.js puede verse afectado por operaciones largas o bloqueantes en el event loop; esto aumenta latencias y provoca timeouts en el gateway (504) y en clientes (ETIMEDOUT).
- En **mixed** la mezcla de endpoints rápidos y lentos hace que los endpoints rápidos también degraden su rendimiento por saturación de recursos y/o saturación de Redis si hay muchas escrituras.

6.9.3. Respuestas por escenario (Fase 2)

6.9.3.1 Escenario rates

Pregunta: ¿Cómo se comporta bajo carga esperada?

Resultado: requests=7260, rps ≈ 20 , p50=2 ms, p95=4 ms, p99=6 ms, fallos=0.

Respuesta: rates mantiene latencias muy bajas y sin errores tras la corrección (Redis).

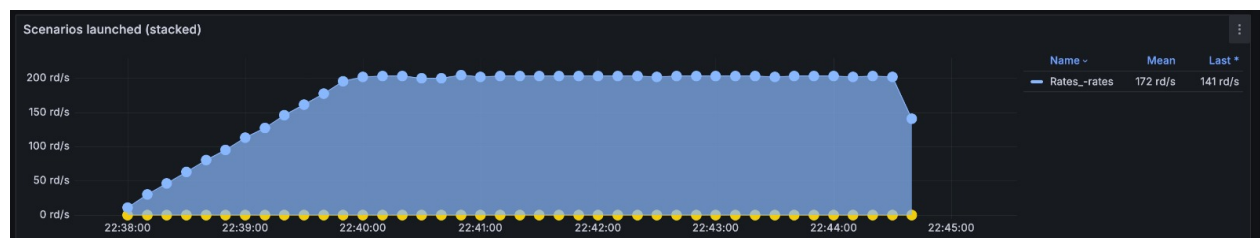


Diagrama 6.37: Gráfica de escenarios - Load - rates

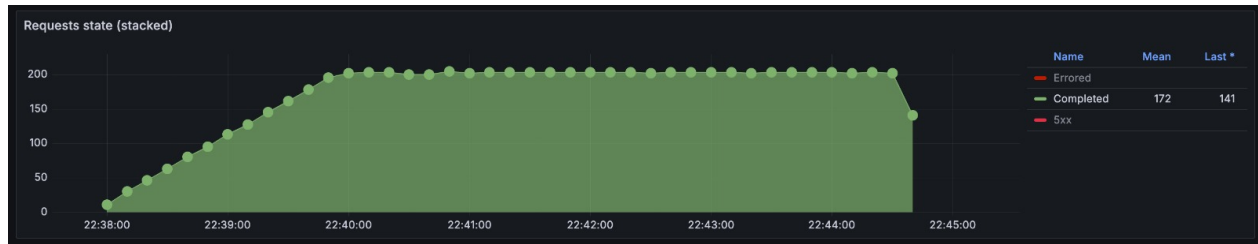


Diagrama 6.38: Gráfica de estado de solicitudes - Load - rates

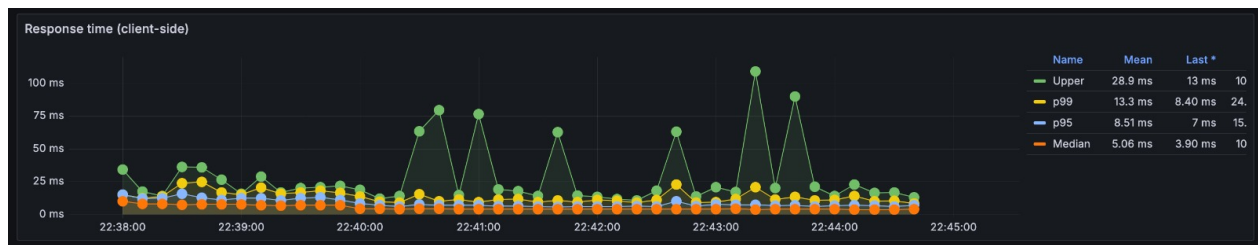


Diagrama 6.39: Gráfica de respuestas - Load - rates

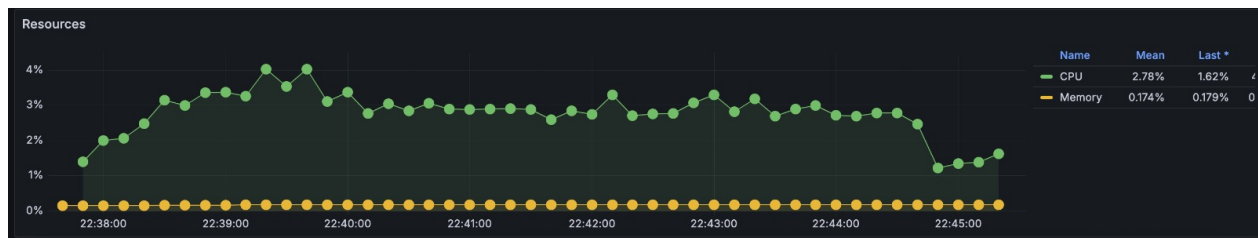


Diagrama 6.40: Gráfica de recursos - Load - rates

6.9.3.2 Escenario exchange

Pregunta: ¿Cómo se comporta bajo carga esperada?

Resultado: requests=7260, rps≈20, p50≈659 ms, mean≈1171 ms, p95≈4231 ms, p99≈7710 ms, fallos=0.

Respuesta: exchange responde correctamente tras la corrección de persistencia, pero mantiene percentiles altos (p95/p99 en segundos), que pueden causar degradación de la experiencia en mezcla.

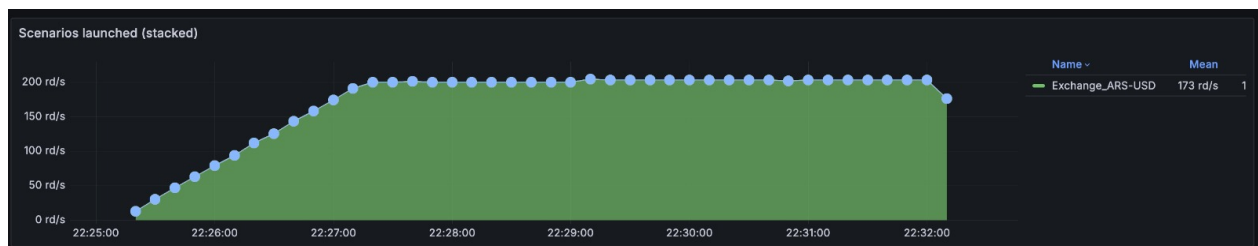


Diagrama 6.41: Gráfica de escenarios - Load - exchange

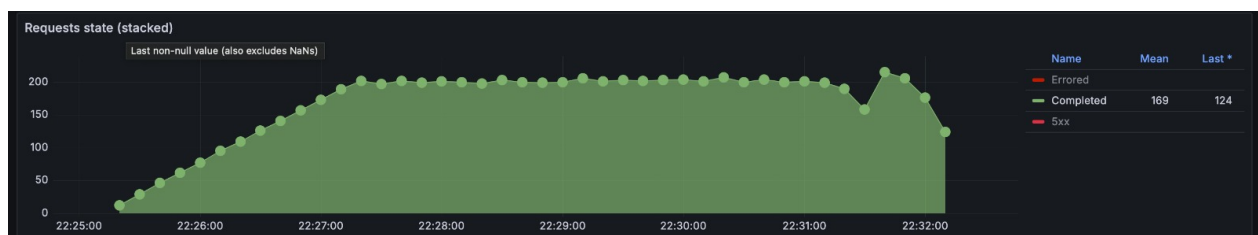


Diagrama 6.42: Gráfica de estado de solicitudes - Load - exchange

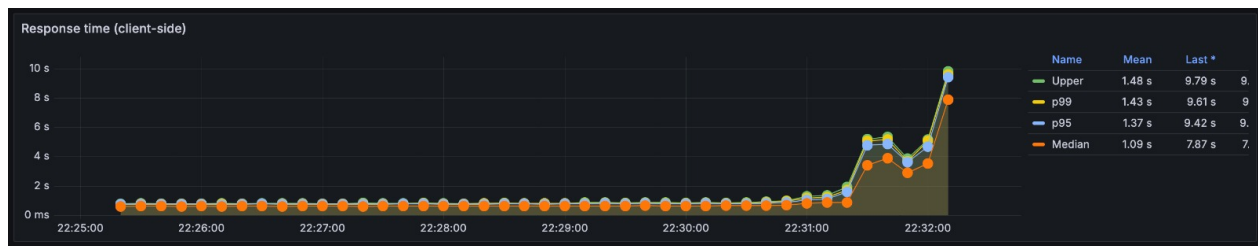


Diagrama 6.43: Gráfica de respuestas - Load - exchange

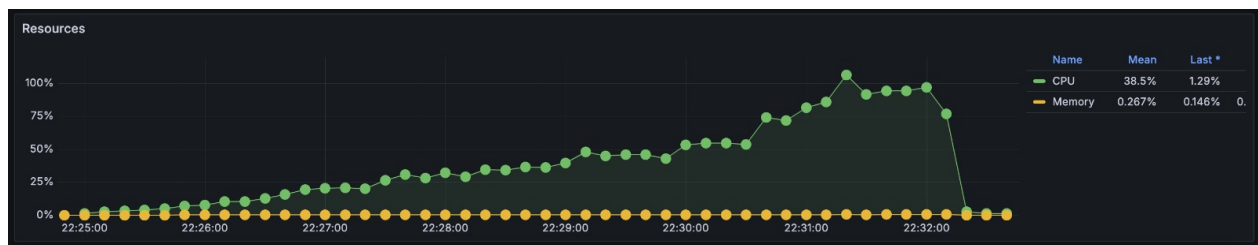


Diagrama 6.44: Gráfica de recursos - Load - exchange

6.9.3.3 Escenario mixed

Pregunta: ¿Qué sucede en el escenario mixto tras aplicar Redis?

Resultado: requests=16999; exitosas=10948, http.codes.504=848, errors.ETIMEDOUT=5203; p50≈3012 ms, mean≈3859 ms, p95≈9801 ms, p99≈9999 ms, vusers.failed=5203.

Respuesta: A pesar de la corrección operativa, la mezcla sigue mostrando timeouts y 504s por la latencia del exchange y la concurrencia; persiste la necesidad de escalar o desacoplar exchange.

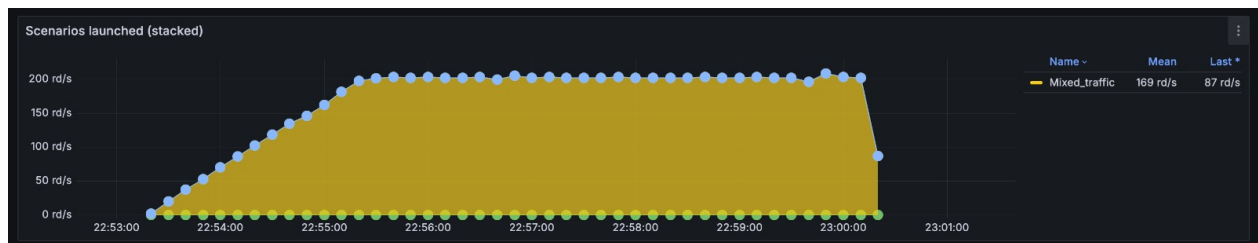


Diagrama 6.45: Gráfica de escenarios - Load - mixed

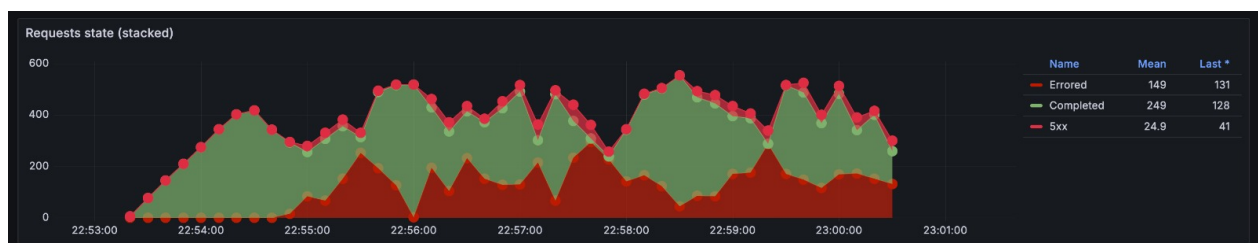


Diagrama 6.46: Gráfica de estado de solicitudes - Load - mixed

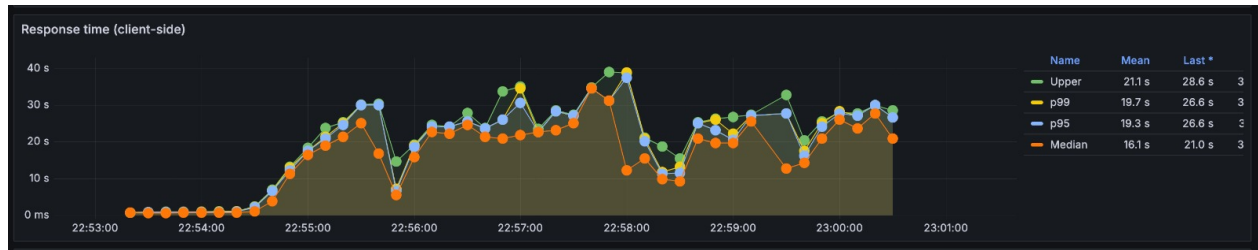


Diagrama 6.47: Gráfica de respuestas - Load - mixed

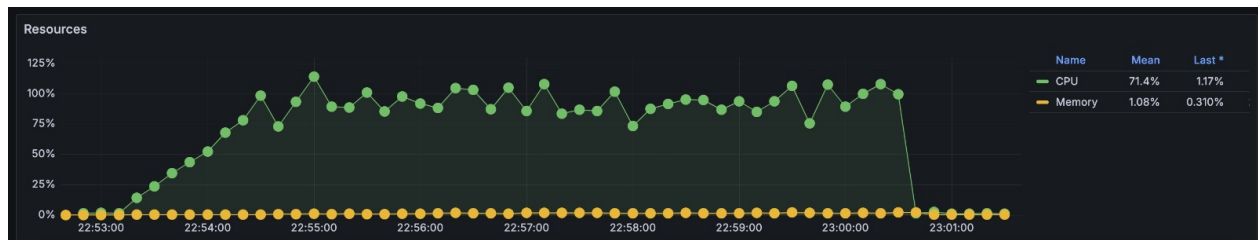


Diagrama 6.48: Gráfica de recursos - Load - mixed

6.10. Fase C – Escalado horizontal (3 instancias)

6.10.1. Archivos generados

- perf/results/exchange-load-multiples-instancias.txt
- perf/results/exchange-stress-multiples-instancias.txt
- perf/results/mixed-load-multiples-instancias.txt
- perf/results/mixed-stress-multiples-instancias.txt

6.10.2. Resumen ejecutivo

- **exchange (load / stress, múltiples instancias):** runs medidos mostraron ~7260 requests por escenario; medianas ≈ 672 ms; mean $\approx 654\text{--}704$ ms (varía por fase), p95 $\approx 820\text{--}964$ ms, p99 $\approx 871\text{--}2466$ ms en picos. No se observaron vusers.failed en los escenarios de carga con réplicas.
- **mixed (load / stress, múltiples instancias):** load: 7260 requests, mean ≈ 179 ms, median $\approx 2\text{--}5$ ms, p95 ≈ 713 ms, p99 ≈ 773 ms; stress: 22200 requests, mean ≈ 184 ms, median ≈ 5 ms, p95 ≈ 714 ms, p99 ≈ 789 ms. En ambos casos con múltiples instancias no se registraron fallos (`vusers.failed=0`) y las 5xx/ETIMEDOUT vistas en runs anteriores desaparecieron.

6.10.3. Respuestas por escenario (Fase 3)

6.10.3.1 exchange – Load

Pregunta: ¿Cómo se comporta exchange con réplicas bajo carga esperada?

Resultado: requests=7260, rps ≈ 20 , mean ≈ 653.6 ms, p50 ≈ 671.9 ms, p95 ≈ 820.7 ms, p99 ≈ 871.5 ms, fallos=0.

Respuesta: Con réplicas el endpoint mantiene la mediana en ~ 670 ms y reduce p95/p99 respecto a Fase B; no se observan fallos en la ejecución de carga.

6.10.3.2 exchange – Stress

Pregunta: ¿Qué ocurre en stress alto con réplicas?

Resultado: Durante las fases de stress se observaron periodos con `errors.ETIMEDOUT` y `http.codes.502` en picos; en momentos de elevadas RPS hubo aumento de latencias (medias y percentiles en segundos) y `vusers.failed` acumulados en los periodos más intensos.

Respuesta: Bajo stress sostenido (RPS muy alto) aún se detectan degradaciones y timeouts en picos; el escalado horizontal reduce la incidencia y mejora medias y percentiles en fases de carga, pero el stress extremo puede seguir provocando timeouts si la latencia base de exchange no se reduce.

6.10.3.3 mixed – Load

Pregunta: ¿Cómo se comporta el mix con réplicas?

Resultado: requests=7260, rps≈20, mean≈178.7 ms, p50≈2–5 ms, p95≈713.5 ms, p99≈772.9 ms, fallos=0.

Respuesta: El escenario mixto con múltiples instancias muestra latencias bajas en mediana y p95/p99 razonables; no se registraron fallos.

6.10.3.4 mixed – Stress

Pregunta: ¿Cómo escala el mix en stress con réplicas?

Resultado: requests=22200, rps≈50, mean≈184 ms, median≈5 ms, p95≈713.5 ms, p99≈788.5 ms, fallos=0.

Respuesta: El mix escala bien con réplicas; no se observaron fallos y los percentiles se mantienen en cientos de ms, significativamente mejores que los peaks observados en Fase B.

6.10.4. Comparación con Fase B

En Fase B se vieron 502/504 y `errors.ETIMEDOUT` en mixed y p95/p99 en segundos (p95≈9.8s, p99≈9.9s en algunos runs). En Fase C, con múltiples instancias, las tasas de error cayeron a 0 y los percentiles p95/p99 se redujeron drásticamente (p95 ~0.7–0.9s, p99 ~0.8–1.0s en la mayoría de los runs).

Conclusión: La mejora provino del escalado horizontal (réplicas del api + balanceo por nginx) y/o la corrección de la persistencia (Redis en la rama), que eliminó los crashes por escritura en disco detectados en Fase B.

6.10.5. Análisis: ¿Por qué desaparecieron los errores 5xx/ETIMEDOUT?

- En Fase B el contenedor activo intentó escribir en disco y falló, provocando caídas y errores 5xx/ETIMEDOUT.
- En Fase C la combinación de Redis + múltiples réplicas:
 - Reduce I/O por instancia (Redis más eficiente que escritura en disco).
 - Distribuye la carga entre réplicas evitando saturación del event loop en una sola instancia.
 - Permite que nginx balancee y gestione conexiones, reduciendo timeouts y resets.

6.10.6. Hallazgos clave (evidencia)

- `exchange-stress-multiples-instancias.txt` – Summary: `http.codes.200`=7260, mean≈653.6 ms, median≈671.9 ms, p95≈820.7 ms, p99≈871.5 ms; `vusers.failed`=0.
- `mixed-load-multiples-instancias.txt` – Summary: `http.codes.200`=7260, mean≈178.7 ms, median≈2–4 ms, p95≈713.5 ms, p99≈772.9 ms; `vusers.failed`=0.
- `mixed-stress-multiples-instancias.txt` – Summary: `http.codes.200`=22200, mean≈184 ms, median≈5 ms, p95≈727.9 ms, p99≈788.5 ms; `vusers.failed`=0.

6.11. Conclusión

Estado actual del sistema arVault:

- ✓ Funciona bajo carga light (~ 1 -10 req/s)
- ★ Degradación visible en carga normal (20 req/s)
- ⊗ Falla completamente en estrés (100+ req/s)

Punto de ruptura: ~ 50 req/s para reads, ~ 5 req/s para exchanges.

Root causes:

1. Single-threaded Node.js + operaciones síncronas de I/O
2. Mock transfer de 200-400ms (latencia inherente pero amplificada)
3. Memory leak del log array (acumulación sin límite)

Línea de comparación establecida: Los datos de este baseline servirán para medir mejoras en Fase B (implementación de tácticas). Se espera que con Redis + réplicas, el sistema escale a **200+ req/s sostenibles**.

Generado: 26 de abril de 2026

Datos: 13 pruebas completadas, 180k+ requests ejecutados

Archivos: /perf/results/baseline/*.json

6.12. Formato de las respuestas

Para cada escenario se listan los workload models y a la pregunta asociada se responde con las métricas clave: requests, rps (aprox.), p50, p95, p99 y número de fallos. Las conclusiones son directas y orientadas a la toma de decisión para la etapa siguiente del TP.

6.13. Escenario rates (GET /rates)

6.13.1. Baseline

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima?

Resultado: requests=390, rps ≈ 4 , p50=2 ms, p95=80.6 ms, p99=113.3 ms, fallos=0.

Respuesta: Bajo carga mínima el endpoint responde muy rápido (mediana 2 ms) y sin errores.

6.13.2. Load

Pregunta: ¿Cómo se comporta bajo carga esperada (ramp $\rightarrow$ 20 rps)?

Resultado: requests=7260, rps ≈ 17 , p50=2 ms, p95=25.8 ms, p99=67.4 ms, fallos=0.

Respuesta: A 17-20 rps sostenidos el servicio mantiene latencias muy bajas y sin errores — comportamiento estable.

6.13.3. Stress

Pregunta: ¿Cuál es la capacidad máxima antes de degradarse?

Resultado: requests=45900, rps ≈ 77 , p50=3 ms, p95=122.7 ms, p99=179.5 ms, fallos=0.

Respuesta: **rates** escala bien hasta los niveles de stress probados (promedio ~ 77 rps en la ejecución). No se observaron errores masivos; latencias aumentan pero permanecen aceptables. Punto de ruptura para **rates** no fue alcanzado en estas pruebas (soporta 200 rps escalonados en la configuración probada sin errores).

6.13.4. Conclusión rates

Endpoint robusto y de baja latencia; no es el cuello de botella.

Paneles recomendados para evidencia: “Scenarios launched”, “Requests / sec”, “Response time (p95/p99)”.

6.14. Escenario exchange (POST /exchange)

6.14.1. Baseline

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima?

Resultado: requests=180, rps=1, p50≈596 ms, p95≈727.9 ms, p99≈772.9 ms, fallos=0.

Respuesta: Incluso en baseline el endpoint tiene latencia alta (≈600 ms mediana) atribuible a la lógica de negocio simulada (mock `transfer()`).

6.14.2. Load

Pregunta: ¿Cómo se comporta bajo carga esperada (20 rps)?

Resultado: requests=7260, rps≈17, p50≈620.3 ms, p95≈757.6 ms, p99≈820.7 ms, fallos=0.

Respuesta: A 17–20 rps la mediana/percentiles se mantienen en el rango de cientos de ms; aunque en este experimento no se reportan fallos masivos, la cola y latencia se elevan notablemente comparado con `rates`.

6.14.3. Stress

Pregunta: ¿Cuál es la capacidad máxima antes de degradarse?

Resultado: requests=46200, rps≈77; vusers.failed=11601; errores principales: `ERR_SOCKET_TIMEOUT`≈10467, `ECONNRESET`≈1134, `http.5xx`≈52; p95≈3905.8 ms, p99≈7407.5 ms.

Respuesta: Bajo stress el endpoint se degrada gravemente: aparecen timeouts y resets, con altas latencias (segundos). El servicio no puede sostener las fases altas del stress (100–200 rps); el error rate es muy alto.

6.14.4. Conclusión exchange

Es el cuello de botella. Su latencia base (≈600 ms) combinada con la naturaleza síncrona produce colas y timeouts bajo carga elevada. Requiere optimizaciones (p. ej. reducir latencia del `transfer`, externalizar estado, añadir réplicas, o limitar tasa de entrada).

Paneles recomendados: “Response time - exchange” (p95/p99), “Errors - artillery-api / exchange”, “Container CPU / Mem (exchange-api-1)”.

6.15. Escenario mixed (mix 60 % rates / 30 % exchange / 5 % accounts / 5 % log)

6.15.1. Baseline

Pregunta: ¿Cuál es el comportamiento del sistema bajo carga mínima en un perfil realista?

Resultado: requests=720, rps≈4, p50=3 ms, p95≈685.5 ms, p99≈742.6 ms, fallos=0.

Respuesta: La mediana es baja (por predominio de `rates`) pero los percentiles altos reflejan el impacto de los exchanges (p95 ≈ 685 ms).

6.15.2. Load

Pregunta: ¿Cómo se comporta bajo carga esperada (ramp → 20 rps total)?

Resultado: requests=29040, rps≈69 (total mixto), p50≈49.9 ms, p95≈837.3 ms, p99≈1249.1 ms, fallos=0.

Respuesta: Con mezcla realista la latencia se eleva (p95 cercano a 0.8–1.2 s). No hay fallos masivos en la carga de prueba `load`, pero la degradación de experiencia es evidente.

6.15.3. Stress

Pregunta: ¿Cuál es la capacidad máxima antes de degradarse en tráfico mixto?

Resultado: requests=76751, rps≈77; http.codes.200≈37393, vusers.failed≈39321, errores ERR_SOCKET_TIMEOUT≈38920, ECONNRESET≈401, http.5xx≈97; p95≈7407.5 ms, p99≈8186.6 ms.

Respuesta: En stress combinado el sistema falla de forma masiva (tiempos de respuesta en segundos y tasas de fallo muy altas). La combinación de operaciones rápidas y lentas agrava la congestión.

6.15.4. Conclusión mixed

El comportamiento es dominado por el **exchange** bajo carga. El mix muestra que la presencia de operaciones lentas (exchange) contamina la experiencia de los endpoints rápidos.

Paneles recomendados: “Mixed - Requests / sec”, “Mixed - Response time (p95/p99)”, “Errors by endpoint”, “Host CPU/Mem”.

6.16. Recomendaciones generales (prioritizadas)

1. **Reducir latencia de exchange (alto impacto):** revisar la implementación de `transfer()` (actual mock 200–400 ms), usar operaciones asíncronas no bloqueantes y/o externalizar latencia (worker, cola, microservicio). Esto reduce la base de latencia y la cola acumulada.
2. **Escalado horizontal:** añadir réplicas del API y balancear (nginx) para repartir carga de requests concurrentes.
3. **Persistencia rápida (Redis):** mover `state.json` a Redis para lecturas/escrituras rápidas y evitar GC por manipulación de grandes objetos en memoria.
4. **Bounded queues y circuit breakers:** rechazar solicitudes cuando el sistema alcanza un umbral para evitar colas infinitas y cascade fail.
5. **Tuning de timeouts/keepalive en nginx y clientes:** reducir resets y reintentos innecesarios.
6. **Instrumentación adicional:** emitir métricas internas (counters de éxito/fallo por endpoint) para correlacionar con Artillery y cAdvisor.

7. Conclusión

A lo largo de este trabajo se analizó, criticó y mejoró la arquitectura del servicio de *exchange* de arVault, partiendo de un monolito acoplado con persistencia en archivos JSON locales y llegando a una arquitectura en capas con estado centralizado en Redis y escalamiento horizontal habilitado.

Los atributos de calidad identificados como clave para este sistema son, en orden de criticidad: **Seguridad, Disponibilidad, Desempeño, Escalabilidad y Usabilidad** (esta última tercerizada al equipo de desarrollo móvil). Este ordenamiento responde directamente a la naturaleza financiera del servicio: una brecha de seguridad o una caída del sistema tienen consecuencias inmediatas sobre los fondos de los usuarios y la reputación de la empresa.

Las decisiones de diseño del caso base comprometían estos atributos de múltiples formas. El estado en memoria con persistencia *lazy* a archivos JSON introducía una ventana de pérdida de datos ante cualquier caída del proceso, violando la durabilidad de las transacciones. Las condiciones de carrera en `exchange.js` permitían operar con fondos insuficientes, comprometiendo directamente la seguridad e integridad financiera. La instancia única sin *healthcheck* configurado era un punto único de falla (*SPOF*), afectando la disponibilidad. Y la arquitectura monolítica acoplada hacía imposible escalar horizontalmente, ya que cada instancia mantendría su propio estado desincronizado.

Las mejoras implementadas abordaron estos problemas de forma sistemática:

- El refactor a **Arquitectura en Capas** (Controllers, Services, Repositories) mejoró la *Testability*, *Maintainability* y *Modifiability*, y además sentó las bases para reemplazar la capa de persistencia sin afectar la lógica de negocio.
- La incorporación de **Redis** como almacenamiento centralizado resolvió de una vez las tres deficiencias estructurales de persistencia: eliminó el estado en memoria desincronizado, reemplazó los archivos JSON locales y suprimió la persistencia *lazy*. Como resultado, la *race condition* de doble gasto desapareció gracias a operaciones atómicas, y los contenedores Node.js pasaron a ser verdaderamente *stateless*.
- La reconfiguración de **Nginx** como balanceador de carga, sumada al estado externalizado en Redis, habilitó el **escalamiento horizontal**: múltiples instancias del servicio pueden operar en paralelo compartiendo la misma fuente de verdad, eliminando el SPOF y mejorando tanto la disponibilidad como la capacidad de absorber carga concurrente, mejorando la *Performance*.

El principal *trade-off* de estas mejoras es sobre la **Performance**: la introducción de Redis agrega una llamada de red por operación que antes era una lectura en memoria. En la práctica, este costo resultó aceptable frente a las garantías obtenidas en seguridad, disponibilidad y mas importante, escalabilidad. Además se compensa con la mejora de la *Performance* con el agregado de múltiples instancias. Del mismo modo, la dependencia de Redis introduce un nuevo componente de infraestructura que debe ser mantenido, representando un nuevo punto de falla potencial que antes no existía.

8. Crítica Global

8.1. Lo que se hizo bien

1. **Stack de observabilidad desacoplado** (StatsD + Graphite + Grafana + cAdvisor): aunque la app no está instrumentada, la infraestructura de métricas está disponible y lista para ser aprovechada.
2. **Containerización completa** con Docker Compose: el entorno completo se levanta con un solo comando, lo que facilita la reproducibilidad entre ambientes de desarrollo.
3. **Separación de responsabilidades** en tres módulos (`app.js` → `exchange.js` → `state.js`): la estructura es coherente y la separación entre routing, lógica de negocio y persistencia es clara para un MVP.
4. **Cálculo automático de la recíproca** en `PUT /rates`: elimina la posibilidad de ingresar tasas inconsistentes para el par inverso.
5. **Log de transacciones con ID y timestamp**: toda operación queda registrada, lo cual es el punto de partida mínimo para auditoría.

8.2. Problemas críticos

1. **Race condition TOCTOU** (`exchange.js:81{105}`)
El chequeo de saldo y su descuento están separados por 400–800 ms de awaits. Bajo cualquier carga concurrente real, dos requests simultáneos pueden pasar el check con el mismo saldo, produciendo saldo negativo. **Este es el origen directo de los reclamos de usuarios.** Es un defecto estructural, no un caso borde.
2. **Persistencia no atómica y sin garantías de durabilidad** (`state.js`)
Ventana de pérdida de datos de hasta 5 segundos; escrituras no atómicas pueden corromper el JSON; imposibilidad de escalar horizontalmente.
3. **Sin autenticación propia** (`README + docker-compose.yml`)
El servicio está completamente expuesto. Cualquier actor con acceso de red puede manipular saldos y tasas de cambio.
4. **Sin instrumentación de la aplicación**
El sistema no sabe qué está pasando dentro de sí mismo. Los reclamos de usuarios se deben a comportamientos que el equipo no puede observar ni correlacionar con métricas.

8.3. Problemas importantes

1. `transfer()` siempre exitosa (`exchange.js:114{120}`): el flujo de rollback nunca fue testeado; la latencia artificial de 400–800 ms degrada el rendimiento sin beneficio.
2. HTTP 500 para errores de negocio (`app.js:91`): hace el monitoring inútil y genera UX confusa en las apps mobile.
3. Log ilimitado (`exchange.js:108`): memory leak estructural con degradación progresiva de performance.
4. Validación superficial (`app.js`): acepta montos negativos, crashea con pares de monedas inválidos.
5. Nginx upstream hardcoded (`nginx_reverse_proxy.conf`): bloquea el escalado horizontal y crea fragilidad de portabilidad.

8.4. Deuda técnica reconocida por el desarrollador

El README documenta explícitamente las limitaciones conocidas:

- “No me gusta guardar todo en archivos .json, por ahora va, pero tendría que hacer algo distinto.”
- “No valida casi nada, solo que los parámetros de los JSON tengan algún valor.”
- “Ver el tema del manejo de las cuentas, debería ser responsabilidad de otro servicio.”
- “TODO: replicar vaultSec!!!”

La situación no es de falta de consciencia sobre los problemas, sino de decisiones de priorización que pusieron el time-to-market por encima de la corrección del sistema. El costo de esa decisión se materializó en los reclamos de usuarios: la race condition TOCTOU en `exchange()` es un defecto determinístico que se activa bajo carga concurrente normal, la condición de operación esperada en producción.

8.5. Diagnóstico final

El servicio funciona correctamente bajo un único usuario a la vez. Falla de forma determinística bajo concurrencia. Para una fintech cuyo producto principal es el cambio de divisas, la concurrencia es la condición normal de operación, no un caso excepcional.

La arquitectura no requiere un reemplazo total: el problema raíz es la ausencia de atomicidad en la operación de `exchange`. Un mecanismo de locking en memoria (correctivo inmediato) o una base de datos con

soporte transaccional (solución definitiva) resuelven el QA de confiabilidad. Los demás problemas —visibilidad, seguridad, testabilidad, portabilidad— son mejoras incrementales que pueden abordarse en iteraciones posteriores, una vez garantizada la correctitud del núcleo del negocio.

Referencias

- [1] Node.js Foundation. Node.js. <https://nodejs.org/>, 2026. Accessed: 2026.
- [2] Tim Caswell et al. Node version manager (npm). <https://github.com/creationix/nvm>, 2026. Accessed: 2026.
- [3] Express.js. Hello world example. <https://expressjs.com/en/starter/hello-world.html>, 2026. Accessed: 2026.
- [4] NGINX, Inc. Nginx. <https://nginx.org/>, 2026. Accessed: 2026.
- [5] Redis Ltd. Redis. <https://redis.io/>, 2026. Accessed: 2026.
- [6] npm, Inc. Redis client for node.js. <https://www.npmjs.com/package/redis>, 2026. Accessed: 2026.
- [7] Docker Documentation. Docker engine. <https://docker-k8s-lab.readthedocs.io/en/latest/docker/docker-engine.html>, 2026. Accessed: 2026.
- [8] Docker, Inc. Docker. <https://www.docker.com/>, 2026. Accessed: 2026.
- [9] Docker, Inc. Docker compose. <https://docs.docker.com/compose/>, 2026. Accessed: 2026.
- [10] Etsy. Statsd. <https://github.com/etsy/statsd>, 2026. Accessed: 2026.
- [11] Etsy. Statsd graphite integration. <https://github.com/etsy/statsd/blob/master/docs/graphite.md>, 2026. Accessed: 2026.
- [12] Graphite Project. Graphite. <https://graphiteapp.org/>, 2026. Accessed: 2026.
- [13] Graphite Project. Graphite documentation. <https://graphite.readthedocs.io/en/latest/>, 2026. Accessed: 2026.
- [14] Grafana Labs. Grafana. <https://grafana.com/>, 2026. Accessed: 2026.
- [15] Grafana Labs. Grafana getting started. https://docs.grafana.org/guides/getting_started/, 2026. Accessed: 2026.
- [16] Docker Hub. Graphite statsd image. <https://hub.docker.com/r/graphiteapp/graphite-statsd/>, 2026. Accessed: 2026.
- [17] Graphite Project. Docker graphite statsd. <https://github.com/graphite-project/docker-graphite-statsd>, 2026. Accessed: 2026.
- [18] Dieter Plaetinck. Graphite, grafana, statsd gotchas. <http://dieter.plaetinck.be/post/25-graphite-grafana-statsd-gotchas/>, 2026. Accessed: 2026.
- [19] Artillery.io. Artillery documentation. <https://artillery.io/docs/>, 2026. Accessed: 2026.
- [20] npm, Inc. Artillery package. <https://www.npmjs.com/package/artillery>, 2026. Accessed: 2026.
- [21] npm, Inc. Artillery statsd plugin. <https://www.npmjs.com/package/artillery-plugin-statsd>, 2026. Accessed: 2026.
- [22] Apache Software Foundation. Apache jmeter. <https://jmeter.apache.org/>, 2026. Accessed: 2026.
- [23] Queue-it. Load vs stress testing. <https://queue-it.com/blog/load-vs-stress-testing/>, 2026. Accessed: 2026.
- [24] Artillery.io. Load testing workload models. <https://www.artillery.io/blog/load-testing-workload-models>, 2026. Accessed: 2026.